

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP ĐỒ ÁN CHUYÊN NGÀNH
HỌC KỲ I, NĂM HỌC 2025-2026

**XÂY DỰNG HỆ THỐNG QUẢN LÝ
CHƯƠNG TRÌNH HỌC BỔNG
DÀNH CHO SINH VIÊN KHÓ KHĂN**

Giảng viên hướng dẫn:
ThS.GVC Phan Thị Phương Nam

Sinh viên thực hiện:
Họ tên: Huỳnh Phước Thọ
MSSV: 110122169
Lớp: DA22TTC

Vĩnh Long, tháng 12 năm 2025

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP ĐỒ ÁN CHUYÊN NGÀNH
HỌC KỲ I, NĂM HỌC 2025-2026

**XÂY DỰNG HỆ THỐNG QUẢN LÝ
CHƯƠNG TRÌNH HỌC BỔNG
DÀNH CHO SINH VIÊN KHÓ KHĂN**

Giảng viên hướng dẫn:
ThS.GVC Phan Thị Phương Nam

Sinh viên thực hiện:
Họ tên: Huỳnh Phước Thọ
MSSV: 110122169
Lớp: DA22TTC

Vĩnh Long, tháng 12 năm 2025

This image shows a full page of a document template designed for handwriting practice. It consists of approximately 30 evenly spaced horizontal dotted lines across the entire width of the page, providing a guide for letter height and placement. The background is plain white, and there are no margins, text, or other markings present.

Giáo viên hướng dẫn
(Ký tên và ghi rõ họ tên)

[illegible]

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn sâu sắc đến giáo viên hướng dẫn cô Phan Thị Phương Nam – giảng viên Khoa Công nghệ Thông Tin. Thời gian vừa qua Cô đã tận tình chỉ bảo, định hướng đề tài và sửa chữa những thiếu sót cho em ngay từ những ngày đầu tiên. Sự hướng dẫn của cô không chỉ giúp em hoàn thiện hệ thống quản lý học bổng trong quá trình thực hành nghiên cứu đồ án chuyên ngành “Xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn” mà còn giúp em học hỏi thêm nhiều kiến thức chuyên môn và kỹ năng thực tế quý giá

Em cũng xin gửi lời tri ân đến Ban lãnh đạo Trường Kỹ thuật và Công nghệ, cùng toàn thể quý Thầy, Cô trong Khoa Công nghệ Thông tin. Những kiến thức nền tảng vững chắc mà quý Thầy, Cô đã truyền đạt trong suốt thời gian học tập tại trường chính là hành trang quan trọng giúp em thực hiện đồ án này.

Trong quá trình nghiên cứu và hoàn thành đề tài, do kiến thức còn hạn chế nên em không tránh được những thiếu sót khi nghiên cứu và tìm hiểu đề tài. Em rất mong sẽ nhận được đóng góp ý kiến của thầy/cô để em có thể khắc phục và hoàn thiện hơn.

Em xin chân thành cảm ơn !

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	i
NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG.....	ii
LỜI CẢM ƠN	iii
MỤC LỤC.....	iv
DANH MỤC HÌNH ẢNH	vii
DANH MỤC BẢNG BIỂU	viii
TÓM TẮT ĐỒ ÁN CHUYÊN NGÀNH	10
MỞ ĐẦU.....	11
1. Mô tả:	11
CHƯƠNG 1 TỔNG QUAN.....	14
1.1 Lý do chọn đề tài	14
1.2 Những vấn đề cần giải quyết	14
1.3 Các nội dung nghiên cứu chính	15
CHƯƠNG 2 NGHIÊN CỨU LÝ THUYẾT	16
2.1 Tìm hiểu về RESTful API	16
2.1.1 Tổng quan về RESTful API.....	16
2.1.2 Phương thức HTTP (HTTP Methods)	17
2.1.3 Trạng thái.....	17
2.1.4 Ưu điểm của RESTful API	18
2.1.5 Nhược điểm của RESTful API	19
2.2 Công nghệ Frontend.	19
2.2.1 ReactJS.....	19
2.2.2 Tailwind CSS	20
2.2.3 Giới thiệu về Tailwind CSS.....	20
2.3 Các công nghệ Backend	22
2.3.1 Nền tảng Node.js.	22
2.3.2 Web Framework Express.js.	25
2.3.3 [1]Các tính năng của Epress.js.....	25
2.4 Cơ sở dữ liệu và ORM.....	26
2.4.1 Hệ quản trị cơ sở dữ liệu MySQL	26
2.4.2 ORM	28

2.5 Cơ chế xác thực bảo mật	31
2.5.1 Xác thực người dùng với JSON Web Token (JWT)	31
2.5.2 Mã hóa và bảo mật dữ liệu	31
CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU	33
3.1 Mô tả bài toán	33
3.2 Mô hình chức năng	34
3.3 Phân tích yêu cầu hệ thống	35
3.3.1 Yêu cầu chức năng	35
3.3.2 Yêu cầu phi chức năng	36
3.4 Thiết kế hệ thống	37
3.4.1 Sơ đồ Usecase	37
3.4.2 Thiết kế cơ sở dữ liệu	40
3.5 Thiết kế giao diện	48
3.5.1 Màn hình Low-fi prototypes trang chủ	48
3.5.2 Màn hình Low-fi prototypes danh sách học bổng	50
3.5.3 Màn hình Low-fi prototypes chi tiết học bổng	51
3.5.4 Màn hình Low-fi prototypes danh sách các trường	53
3.5.5 Màn hình Low-fi prototypes hồ sơ cá nhân	54
3.5.6 Màn hình Low-fi prototypes hồ sơ nộp học bổng của sinh viên	56
CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU	58
4.1 Tổng quan kết quả	58
4.2 Giao diện người dùng	59
4.2.1 Giao diện trang chủ	59
4.2.2 Giao diện học bổng	59
4.2.3 Giao diện danh sách các trường	60
4.2.4 Giao diện hướng dẫn	61
4.2.5 Giao diện học bổng	62
4.2.6 Giao diện nộp học bổng	63
4.2.7 Giao diện hồ sơ sinh viên	64
4.2.8 Giao diện hồ sơ đã nộp	65
4.2.9 Giao diện thông tin cá nhân sinh viên	66
4.3 Kết quả đạt được	66

4.3.1 Đánh giá chung	68
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	69
5.1 Kết luận chung.....	69
5.2 Hạn chế	70
5.3 Hướng phát triển.....	70
DANH MỤC TÀI LIỆU THAM KHẢO	72

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ hoạt động của RESTful API	16
Hình 2.2: Các tính năng cơ bản của nodejs.....	23
Hình 2.3: Kiến trúc hoạt động của Node.js.....	23
Hình 2.3: Cách hoạt động của MySQL.....	28
Hình 2.4: Mô hình Object–Relational Mapping (ORM).....	29
Hình 3.1 Mô hình chức năng.....	35
Hình 3.2: Mô hình Usecase	37
Hình 3.3: Mô hình dữ liệu quan hệ	40
Hình 3.4: Mô hình dữ liệu vật lý	41
Hình 3.5: Lược đồ cơ sở dữ liệu	43
Hình 3.6: Màn hình Low-fi prototypes trang chủ	49
Hình 3.7: Màn hình Low-fi prototypes danh sách học bổng	50
Hình 3.8: Màn hình Low-fi prototypes danh sách các trường	53
Hình 4.1 : Giao diện trang chủ	59
Hình 4.2: Giao diện học bổng.	60
Hình 4.3: Giao diện danh sách các trường.....	61
Hình 4.4: Giao diện hướng dẫn.....	62
Hình 4.5: Giao diện học bổng	62
Hình 4.6: Giao diện nộp học bổng	63
Hình 4.7: Giao diện hồ sơ sinh viên.....	64
Hình 4.8: Giao diện hồ sơ đã nộp.....	65
Hình 4.9: Giao diện thông tin cá nhân sinh viên.....	66

DANH MỤC BẢNG BIỂU

Bảng 3.1. Bảng mô tả các Actor.	38
Bảng 3.2. Bảng mô tả các usecase.	38
Bảng 3.3. Mô tả bảng User.....	44
Bảng 3.4. Mô tả bảng School.....	44
Bảng 3.5. Mô tả bảng Student.....	45
Bảng 3.6. Mô tả bảng Sponsor.....	46
Bảng 3.7. Mô tả bảng Scholarship.....	46
Bảng 3.8. Mô tả bảng Application.....	47

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Giải thích
1	API	Application Programming Interface
2	REST	Representational State Transfer
3	JWT	JSON Web Token
4	ORM	Object Relational Mapping
5	CSDL	Cơ sở dữ liệu
6	MySQL	My Structured Query Language
7	RBAC	Role-Based Access Control
8	HTTP	HyperText Transfer Protocol
9	HTTPS	HyperText Transfer Protocol Secure
10	RDBMS	Relational Database Management System
11	SPA	Single Page Application
12	MVC	Model - View - Controller
13	NPM	Node Package Manager

TÓM TẮT ĐỒ ÁN CHUYÊN NGÀNH

Dự án "Xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn" tập trung vào việc số hóa toàn diện quy trình cấp phát học bổng, tạo ra một nền tảng kết nối minh bạch giữa Nhà trường, Nhà tài trợ và Sinh viên. Vấn đề đặt ra là làm thế nào để thiết kế một hệ thống đa trường (Multi-tenant) có khả năng xử lý quy trình xét duyệt phức tạp, đảm bảo tính công bằng, chống gian lận dữ liệu và bảo mật thông tin cá nhân nhạy cảm.

Hướng tiếp cận đề tài:

- Nghiên cứu lý thuyết: Tìm hiểu các công nghệ cốt lõi như RESTful API, kiến trúc hệ thống Multi-tenant (Đa đơn vị), cơ chế xác thực và phân quyền người dùng (JWT, RBAC) và nghiên cứu cách hoạt động của ReactJS cho phía Client, Node.js và Express.js cho phía Server, cùng với ORM Sequelize để quản trị cơ sở dữ liệu quan hệ MySQL.

- Nghiên cứu thực nghiệm: Xây dựng từng thành phần hệ thống dựa trên mô hình MVC. Thiết kế Database chuẩn hóa để lưu trữ hồ sơ và lịch sử xét duyệt (Data Snapshot). Phát triển các API xử lý nghiệp vụ phức tạp như: tự động chấm điểm ưu tiên (Auto-ranking), đóng băng dữ liệu khi nộp hồ sơ, và kết xuất báo cáo Excel. Thực nghiệm kiểm thử quy trình từ lúc sinh viên nộp đơn đến khi giải ngân.

Kết quả mong đợi của đề tài:

- Hệ thống website quản lý học bổng hoàn chỉnh với giao diện thân thiện, chuẩn UX/UI, giúp thay thế quy trình nộp hồ sơ giấy thủ công rườm rà.

- Tính năng xét duyệt hồ sơ hoạt động chính xác, hỗ trợ công cụ lọc, sắp xếp và phát hiện gian lận, giúp giảm tải khối lượng công việc cho cán bộ quản lý.

- Đảm bảo tính minh bạch tài chính thông qua các báo cáo thống kê chi tiết dành cho Nhà tài trợ và Nhà trường.

- Hệ thống hoạt động ổn định, bảo mật cao, phân quyền chặt chẽ giữa các đối tượng (Admin, Sinh viên, Nhà tài trợ) và có khả năng mở rộng cho nhiều trường đại học cùng sử dụng.

MỞ ĐẦU

1. Mô tả:

Đề tài tập trung xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn tại các trường đại học. Giúp tương tác giữa nhà trường, các nhà tài trợ và sinh viên dễ dàng hơn từ đó quy trình từ việc công bố học bổng, nộp hồ sơ, xét duyệt cho đến khi trao học bổng trở nên minh bạch, chính xác và hiệu quả.

Hệ thống cho phép theo dõi và quản lý thông tin các gói học bổng, tiếp nhận và xét duyệt hồ sơ trực tuyến của sinh viên. Cung cấp môi trường để tìm kiếm các học bổng phù hợp, nộp hồ sơ, theo dõi trạng thái và nhận thông báo kết quả. Ngoài ra, tạo các báo cáo thống kê nhằm phục vụ công tác quản lý và báo cáo của nhà trường.

Công nghệ sử dụng bao gồm:

- Frontend: ReactJS;
- Backend: Node.js + Express.js;
- ORM (Kết nối Backend với Database): Sequelize;
- Database: MySQL.

2. Phương pháp thực hiện:

– Mục tiêu nghiên cứu:

- + Xây dựng hệ thống tập trung quản lý thông tin giúp giảm thiểu việc quản lý bằng giấy tờ;
- + Tạo ra một kênh thông tin chính thức, tiện lợi cho sinh viên trong việc tìm kiếm và nộp hồ sơ xin học bổng;
- + Tối ưu hóa quy trình xét duyệt hồ sơ cho ban quản trị, đảm bảo tính công bằng, minh bạch và nhanh chóng;
- + Cung cấp giải pháp giúp nhà trường quản lý học bổng dễ dàng theo dõi, tìm kiếm và thống kê thông tin.

– Nội dung nghiên cứu:

- + Tìm hiểu và phân tích quy trình quản lý, xét duyệt học bổng thực tế tại các trường đại học;
- + Xác định các đối tượng tham gia hệ thống:

- + Thiết kế kiến trúc hệ thống và các quy trình nghiệp vụ;
- + Thiết kế mô hình cơ sở dữ liệu quan hệ để lưu trữ thông tin về sinh viên, học bổng, hồ sơ và nhà tài trợ
- + Xây dựng các chức năng của hệ thống:
 - Quản lý thông tin chương trình học bổng và nhà tài trợ;
 - Quản lý hồ sơ đăng ký học bổng của sinh viên;
 - Thực hiện xét duyệt và cập nhật kết quả học bổng;
 - Thống kê và xuất báo cáo.
- **Đối tượng nghiên cứu:**
 - Các quy trình nghiệp vụ liên quan đến việc cấp và quản lý học bổng cho sinh viên có hoàn cảnh khó khăn;
 - Các nhóm người dùng chính:
 - + Admin: Quản trị toàn bộ hệ thống, phân quyền và giám sát dữ liệu;
 - + Cán bộ quản lý: Cập nhật, xét duyệt hồ sơ và xuất báo cáo;
 - + Sinh viên: Đăng ký học bổng, theo dõi tiến trình và nhận thông báo kết quả;
 - + Nhà tài trợ: Xem thống kê học bổng đã cấp và danh sách sinh viên được nhận.
- **Phương pháp nghiên cứu:**

Phương pháp nghiên cứu lý thuyết:

 - + Tham khảo các hệ thống quản lý học bổng hiện nay của các trường đại học, đọc hiểu tài liệu về quy trình xét duyệt học bổng và các tiêu chí dành cho sinh viên có hoàn cảnh khó khăn;
 - + Nghiên cứu tài liệu về ReactJS, Express.js, Sequelize và MySQL để hiểu cách thiết kế giao diện, thiết kế mô hình cơ sở dữ liệu và cách xây dựng API theo mô hình RESTful;
 - + Tìm hiểu các phương pháp phân quyền người dùng, xử lý xác thực bằng JWT, cũng như các tiêu chuẩn bảo mật cơ bản trong phát triển ứng dụng;
 - + Thu thập dữ liệu mẫu về các chương trình học bổng, thông tin sinh viên và nhà tài trợ nhằm phục vụ mục đích tạo dữ liệu demo cho hệ thống.

Phương pháp nghiên cứu thực nghiệm:

- + Cài đặt môi trường lập trình và tiến hành xây dựng giao diện website bằng ReactJS phát triển các chức năng cơ bản như xem danh sách học bổng, xem chi tiết học bổng, đăng ký tài khoản và đăng nhập;
 - + Xây dựng backend bằng Node.js và Express.js để xử lý các API đăng ký học bổng, nộp hồ sơ, xét duyệt và phân quyền người dùng;
 - + Sử dụng Sequelize để kết nối và quản lý dữ liệu trên MySQL, bao gồm thêm, sửa, xóa và truy xuất dữ liệu;
 - + Thực hiện kiểm thử các chức năng đã xây dựng, đánh giá tính đúng đắn và khả năng hoạt động của hệ thống, sau đó hoàn thiện và tối ưu giao diện cũng như chức năng quản trị phục vụ quản lý dữ liệu học bổng.
- **Phạm vi thực hiện đề tài:**
- + Hệ thống tập trung vào các nghiệp vụ chính trong phạm vi một số trường đại học;
 - + Tập trung vào quy trình đăng ký xét duyệt học bổng và báo cáo số liệu.

3. Kết quả đạt được:

- Chức năng quản lý học bổng;
- Sinh viên có thể nộp hồ sơ trực tuyến và theo dõi trạng thái;
- Hệ thống xét duyệt hồ sơ với phân quyền rõ ràng;
- Thống kê và báo cáo theo từng gói học bổng, nhà tài trợ, năm học;
- Liên kết thông tin giữa nhà tài trợ, sinh viên và nhà trường.

CHƯƠNG 1 TỔNG QUAN

1.1 Lý do chọn đề tài

Hiện nay, công tác quản lý và cấp phát học bổng tại nhiều trường đại học vẫn còn thực hiện chủ yếu qua các phương thức thủ công như nộp hồ sơ giấy, quản lý bằng Excel hoặc email rời rạc. Cách làm này bộc lộ nhiều hạn chế tốn kém thời gian, dễ thất lạc hồ sơ, khó khăn trong việc tra cứu lịch sử và đặc biệt là thiếu tính minh bạch đối với các Nhà tài trợ.

Ngoài ra, sinh viên có hoàn cảnh khó khăn thường gặp trở ngại trong việc tiếp cận thông tin các gói học bổng phù hợp. Việc thiếu kênh thông tin chính thống và quy trình nộp đơn trực tuyến khiến nhiều sinh viên bỏ lỡ cơ hội được hỗ trợ.

Do đó, việc xây dựng một hệ thống quản lý học bổng trên nền tảng web hiện đại (sử dụng ReactJS và Node.js) không chỉ giúp tối ưu hóa quy trình hành chính mà còn đảm bảo tính công bằng, nhân văn và minh bạch trong giáo dục.

1.2 Những vấn đề cần giải quyết

Các vấn đề chính mà dự án hướng đến giải quyết bao gồm:

- Số hóa quy trình xét duyệt: Chuyển đổi toàn bộ quy trình từ lúc sinh viên nộp đơn, tải minh chứng, cho đến khi cán bộ xét duyệt và giải ngân sang môi trường trực tuyến. Hệ thống cần đảm bảo xử lý được lượng lớn hồ sơ mà không bị tắc nghẽn hay sai sót dữ liệu;
- Đảm bảo tính công bằng và chống gian lận: Giải quyết bài toán lưu vết dữ liệu tại thời điểm nộp để đảm bảo hồ sơ được xét dựa trên đúng hoàn cảnh thực tế, tránh việc sửa đổi dữ liệu sau khi đã nộp. Đồng thời, hệ thống cần hỗ trợ tính năng tự động xếp hạng để gợi ý các hồ sơ ưu tiên một cách khách quan;
- Quản lý đa trường: Thiết kế kiến trúc hệ thống sao cho nhiều trường đại học có thể cùng sử dụng trên một nền tảng nhưng dữ liệu vẫn được tách biệt hoàn toàn đảm bảo tính bảo mật và riêng tư cho từng đơn vị.
- Minh bạch tài chính: Cung cấp công cụ cho phép Nhà tài trợ theo dõi dòng tiền của mình đã được cấp cho ai, vào thời điểm nào, qua đó gia tăng niềm tin và khuyến khích hoạt động tài trợ giáo dục.

1.3 Các nội dung nghiên cứu chính

Để giải quyết các vấn đề trên, đề tài tập trung nghiên cứu và triển khai các nội dung sau:

- Nghiên cứu công nghệ: Tìm hiểu và ứng dụng các công nghệ lập trình web hiện đại. Sử dụng ReactJS để xây dựng giao diện người dùng tương tác cao (SPA), Node.js & Express.js để xây dựng Backend hiệu năng cao xử lý các nghiệp vụ logic phức tạp. Sử dụng MySQL kết hợp Sequelize ORM để thiết kế và quản trị cơ sở dữ liệu quan hệ.

- Nghiên cứu nghiệp vụ và quy trình: Phân tích sâu quy trình cấp phát học bổng thực tế từ đó xây dựng các luồng chức năng cho 4 nhóm người dùng: Quản trị hệ thống (Super Admin), Cán bộ trường (Uni Admin), Sinh viên (Student) và Nhà tài trợ (Sponsor).

Triển khai các tính năng cốt lõi:

- Xây dựng cơ chế xác thực và phân quyền bảo mật (JWT, RBAC);
- Phát triển module quản lý hồ sơ với tính năng Upload minh chứng;
- Xây dựng module thống kê, báo cáo và xuất dữ liệu ra Excel phục vụ công tác giải ngân.

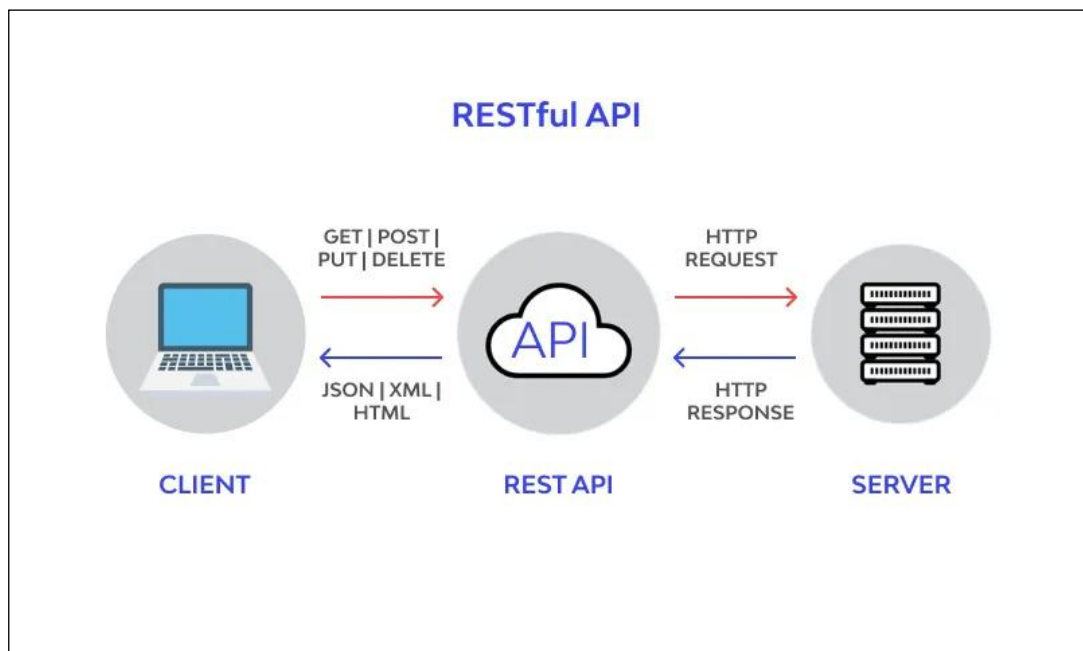
CHƯƠNG 2 NGHIÊN CỨU LÝ THUYẾT

2.1 Tìm hiểu về RESTful API

2.1.1 Tổng quan về RESTful API

RESTful API (Representational State Transfer Application Programming Interface) là một kiểu thiết kế giao diện lập trình ứng dụng dựa trên nguyên tắc REST – một phong cách kiến trúc phần mềm được giới thiệu lần đầu bởi Roy Fielding vào năm 2000. RESTful API sử dụng giao thức HTTP để trao đổi dữ liệu giữa client (frontend) và server (backend), giúp các ứng dụng có thể giao tiếp với nhau một cách hiệu quả, nhanh chóng và dễ dàng mở rộng [1].

RESTful API (Representational State Transfer) là một kiến trúc tiêu chuẩn được sử dụng để thiết kế các giao diện lập trình ứng dụng (API) cho phép các ứng dụng web giao tiếp với nhau một cách hiệu quả và đơn giản. RESTful API tập trung vào việc quản lý các tài nguyên (resources) trên hệ thống, bao gồm các tệp, ảnh, dữ liệu động.



Hình 2.1. Sơ đồ hoạt động của RESTful API

(Nguồn: <https://vietnix.vn/restful-api-la-gi/>)

RESTful API (Representational State Transfer Application Programming Interface) là một kiểu thiết kế API phổ biến trong việc xây dựng các ứng dụng web.

Nó cho phép các ứng dụng (web, mobile, desktop,...) giao tiếp với nhau thông qua giao thức HTTP.

RESTful API hoạt động dựa trên khái niệm **resource** (tài nguyên), trong đó mỗi tài nguyên được xác định bằng một URL duy nhất. Nó sử dụng các phương thức HTTP như GET, POST, PUT, DELETE để thao tác với dữ liệu [1].

2.1.2 Phương thức HTTP (HTTP Methods)

RESTful API sử dụng các phương thức HTTP để xác định hành động cần thực hiện trên tài nguyên:

- GET: Lấy hoặc truy vấn dữ liệu của một hoặc nhiều tài nguyên từ server mà không làm thay đổi trạng thái dữ liệu trên server;
- POST: Tạo mới một tài nguyên trên server. Phương thức này gửi dữ liệu từ client đến server để thêm tài nguyên mới;
- PUT: Cập nhật toàn bộ dữ liệu của một tài nguyên hiện có trên server. Client cần gửi toàn bộ dữ liệu của tài nguyên, không chỉ các trường cần thay đổi;
- DELETE: Xóa tài nguyên cụ thể trên server.

Các phương thức HTTP như GET, POST, PUT, DELETE trong RESTful API không chỉ xác định hành động mà còn giúp API tuân thủ các chuẩn giao tiếp, đảm bảo tính rõ ràng và dễ sử dụng. Việc áp dụng đúng các phương thức này giúp server dễ dàng xử lý các yêu cầu từ client và duy trì hệ thống nhất quán.

2.1.3 Trạng thái

RESTful API sử dụng các mã trạng thái HTTP để thông báo kết quả của các yêu cầu từ client. Các mã trạng thái này giúp client hiểu được tình trạng của yêu cầu và hướng xử lý tiếp theo, mỗi mã trạng thái mang một ý nghĩa cụ thể về kết quả của yêu cầu, có thể là thành công, lỗi phía người dùng, hoặc lỗi hệ thống. Dưới đây là các mã trạng thái HTTP phổ biến mà một RESTful API có thể trả về [1]:

- 200 (OK): Yêu cầu đã được xử lý thành công và server đã trả về kết quả;
- 201 (Created): Tài nguyên đã được tạo thành công trên server;

- 400 (Bad Request): Yêu cầu của client không hợp lệ hoặc thiếu thông tin cần thiết để xử lý;
- 401 (Unauthorized): Client không có quyền truy cập tài nguyên hoặc yêu cầu này yêu cầu phải xác thực;
- 403 (Forbidden): Client đã được xác thực nhưng không có quyền truy cập tài nguyên yêu cầu;
- 404 (Not Found): Tài nguyên yêu cầu không tồn tại hoặc không thể tìm thấy trên server.

Mã trạng thái HTTP giúp xác định rõ ràng kết quả của một yêu cầu và giúp client hiểu được tình trạng của tài nguyên hoặc hành động yêu cầu. Các mã trạng thái này không chỉ phản ánh tình trạng thành công hay lỗi, mà còn giúp client xử lý thông báo lỗi một cách hiệu quả hơn thông qua các phản hồi chi tiết.

2.1.4 Ưu điểm của RESTful API

RESTful API mang lại nhiều ưu điểm, giúp cho việc phát triển và triển khai các ứng dụng trở nên dễ dàng và hiệu quả. Dưới đây là một số ưu điểm chính của RESTful API [1]:

Đơn giản và dễ dùng: Mô hình giao tiếp đơn giản: RESTful API sử dụng HTTP làm giao thức truyền thông, dễ hiểu và đơn giản để triển khai. Các phương thức như GET, POST, PUT, DELETE đều được xác định rõ ràng, giúp việc sử dụng và bảo trì API trở nên dễ dàng hơn;

Khả năng mở rộng: RESTful API dễ mở rộng nhờ vào nguyên lý đơn giản và chuẩn HTTP. Nó hỗ trợ nhiều loại ứng dụng khác nhau, từ web đến mobile, và không gặp vấn đề tương thích giữa các nền tảng hay ngôn ngữ lập trình;

Tính linh hoạt của ứng dụng: RESTful API không phụ thuộc vào trạng thái trước đó, giúp server dễ bảo trì và mở rộng. Nó có thể được sử dụng cho nhiều loại ứng dụng mà không bị hạn chế bởi hệ điều hành hay framework;

Tính tương thích và mở: RESTful API có thể tích hợp và giao tiếp với nhiều hệ thống khác nhau, như cơ sở dữ liệu, dịch vụ web khác, hoặc các hệ thống bên ngoài. Điều này giúp mở rộng khả năng kết nối và tích hợp trong một môi

trường đa dạng. RESTful API được sử dụng rộng rãi, vì vậy cộng đồng phát triển luôn cung cấp tài liệu và hỗ trợ kỹ thuật để giải quyết các vấn đề liên quan đến API.

2.1.5 Nhược điểm của RESTful API

Không hỗ trợ trạng thái (Stateless): Vì RESTful API không lưu trữ trạng thái giữa các yêu cầu mỗi yêu cầu phải chứa toàn bộ thông tin cần thiết để server xử lý. Điều này có thể dẫn đến việc yêu cầu nhiều dữ liệu trong mỗi lần gọi API, làm tăng băng thông và giảm hiệu suất, đặc biệt với các ứng dụng cần duy trì trạng thái người dùng hoặc phiên làm việc [2].

Quá tải khi có quá nhiều yêu cầu: Khi hệ thống yêu cầu nhiều tài nguyên và giao dịch, RESTful API có thể gặp khó khăn trong việc xử lý hàng loạt yêu cầu. Việc phải thực hiện nhiều yêu cầu HTTP liên tiếp có thể gây tải cao cho server và giảm hiệu suất tổng thể của hệ thống .

2.2 Công nghệ Frontend.

2.2.1 ReactJS

2.2.1.1 Tổng quan về ReactJS

ReactJS là thư viện JavaScript chuyên biệt cho việc xây dựng giao diện người dùng động, dựa trên kiến trúc component (thành phần giao diện). Điểm nổi bật của ReactJS nằm ở khả năng quản lý trạng thái (state management) hiệu quả, kết hợp cùng Virtual DOM giúp tối ưu quá trình render giao diện.[3]

2.2.1.2 Các tính năng của ReactJS

ReactJS có nhiều tính năng hữu ích cho việc phát triển ứng dụng web, bao gồm các tính năng [3]:

Components: ReactJS cho phép phát triển ứng dụng web theo mô hình component. Các component là các phần tử UI độc lập có thể được tái sử dụng trong nhiều phần khác nhau của ứng dụng;

Virtual DOM: ReactJS sử dụng Virtual DOM để tối ưu hóa hiệu suất của ứng dụng. Virtual DOM là một bản sao của DOM được lưu trữ trong bộ nhớ và được cập nhật một cách nhanh chóng khi có thay đổi, giúp tăng tốc độ và hiệu suất của ứng dụng;

JSX: JSX là một ngôn ngữ lập trình phân biệt được sử dụng trong ReactJS để mô tả các thành phần UI. JSX kết hợp HTML và JavaScript, giúp cho việc viết mã dễ hiểu và dễ bảo trì hơn;

State và Props: ReactJS cho phép quản lý trạng thái của các thành phần UI thông qua State và Props. State là trạng thái của một thành phần được quản lý bởi nó chính, trong khi Props là các giá trị được truyền vào từ bên ngoài để tùy chỉnh hoặc điều khiển hành vi của một thành phần;

Hỗ trợ tốt cho SEO: ReactJS hỗ trợ tốt cho việc tối ưu hóa SEO. Với các thư viện như React Helmet, các nhà phát triển có thể quản lý các phần tử meta và title cho từng trang web, giúp tăng khả năng tìm kiếm và tăng cường trải nghiệm người dùng;

Hỗ trợ đa nền tảng: ReactJS không chỉ được sử dụng để phát triển ứng dụng web, mà còn được sử dụng để phát triển ứng dụng di động với React Native. Sử dụng React Native, các nhà phát triển có thể xây dựng ứng dụng di động cho cả iOS và Android sử dụng cùng một mã nguồn;

Redux: Redux là một thư viện quản lý trạng thái cho các ứng dụng ReactJS. Nó giúp quản lý trạng thái của ứng dụng một cách chính xác và dễ dàng, đồng thời giúp tăng tính linh hoạt và khả năng mở rộng của ứng dụng.

2.2.2 Tailwind CSS

2.2.3 Giới thiệu về Tailwind CSS

Tailwind CSS là một Utility-first CSS framework giúp lập trình viên xây dựng giao diện web nhanh chóng thông qua việc sử dụng các class CSS dựng sẵn. Thay vì viết CSS thủ công cho từng thành phần, bạn có thể kết hợp các class để định nghĩa trực tiếp giao diện ngay trong HTML [4].

Đặc điểm kỹ thuật của Tailwind CSS:

- Utility-first: Cung cấp hàng trăm class CSS nhỏ gọn để tùy chỉnh giao diện mà không cần viết CSS mới;
- Highly Customizable: Có thể tùy chỉnh theme, màu sắc, font chữ, spacing... thông qua file `tailwind.config.js`;

- Responsive Design: Tích hợp sẵn breakpoint theo mobile-first, dễ dàng tạo layout responsive chỉ bằng class;
- JIT Compiler (Just-In-Time): Tự động biên dịch class khi cần, giúp code gọn nhẹ, giảm dung lượng CSS không sử dụng;
- Performance Optimization: Kết hợp PurgeCSS để loại bỏ CSS thừa, tối ưu tốc độ tải trang;
- Ecosystem và Plugins: Hỗ trợ nhiều plugin mở rộng như typography, forms, aspect-ratio... giúp tăng tính linh hoạt.

2.2.3.1 Ưu điểm của Tailwind CSS

Tailwind CSS mang đến nhiều lợi ích nổi bật, đặc biệt là khả năng tăng tốc phát triển giao diện và tối ưu hiệu suất website [4].

Một số ưu điểm dễ thấy nhất gồm:

- **Tăng tốc độ phát triển giao diện:** Tailwind CSS cung cấp hệ thống class tiện ích (utility-first), giúp lập trình viên không phải viết lại CSS thủ công từ đầu. Nhờ đó, việc xây dựng giao diện trở nên nhanh chóng và đặc biệt phù hợp với những dự án cần ra mắt gấp;
- **Dễ tùy biến và mở rộng:** Framework này cho phép tùy chỉnh theme, màu sắc, font chữ hay khoảng cách trực tiếp trong file tailwind.config.js. Đồng thời, nó hỗ trợ nhiều plugin mở rộng như typography, forms, aspect-ratio... giúp tăng tính linh hoạt cho dự án;
- **Hỗ trợ responsive và dark mode:** Với Tailwind, bạn chỉ cần thêm các prefix như sm:, md:, lg: hoặc dark: để tạo giao diện phù hợp với nhiều thiết bị khác nhau. Điều này giúp lập trình viên tiết kiệm thời gian khi phát triển responsive và hỗ trợ chế độ tối;
- **Cộng đồng lớn, tài liệu phong phú:** Tailwind CSS được đông đảo lập trình viên tin dùng, hình thành cộng đồng rộng lớn với nhiều plugin hữu ích. Bên cạnh đó, tài liệu hướng dẫn chi tiết, dễ tiếp cận giúp người mới bắt đầu học nhanh chóng;

– **Tối ưu hiệu suất website:** Tailwind tích hợp sẵn công cụ PurgeCSS và JIT Compiler, có khả năng loại bỏ CSS thừa và chỉ biên dịch class khi cần thiết. Nhờ đó, file CSS được gọn nhẹ hơn, cải thiện đáng kể tốc độ tải trang.

2.2.3.2 Nhược điểm của Tailwind CSS

Bên cạnh những ưu thế nổi bật, Tailwind CSS cũng có một số hạn chế mà lập trình viên cần lưu ý để tránh rủi ro trong quá trình triển khai [4]:

– **HTML dễ bị dài và khó đọc:** Khi áp dụng quá nhiều class vào một phần tử, đoạn mã HTML có thể trở nên “cồng kềnh” và khó theo dõi. Điều này đôi khi gây khó khăn cho những lập trình viên mới tiếp cận;

– **Đòi hỏi thời gian làm quen:** Người mới phải nhớ hệ thống class khá nhiều (ví dụ: bg-blue-500, px-4, text-lg). Ban đầu có thể cảm thấy phức tạp so với CSS thuần;

– **Khó quản lý khi thiếu quy ước chung:** Nếu team không có guideline, code dễ bị trùng lặp class và khó duy trì về lâu dài;

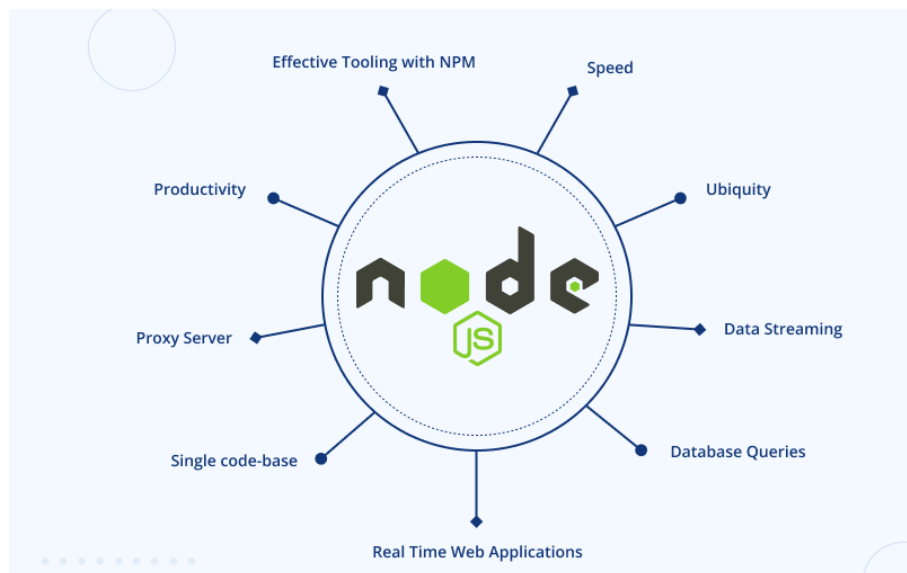
– **Không phù hợp cho dự án nhỏ, đơn giản:** Với website tĩnh, ít trang, việc tích hợp Tailwind có thể “thừa tính năng” so với CSS thuần.

2.3 Các công nghệ Backend

2.3.1 Nền tảng Node.js.

2.3.1.1 Giới thiệu về Node.js

Node.js là một nền tảng xây dựng ứng dụng mạng có hiệu suất cao dựa trên JavaScript runtime của Chrome V8. Nó cho phép lập trình viên sử dụng JavaScript để viết mã cho cả phía server và client, đồng thời cung cấp khả năng xử lý không đồng bộ và lập trình sự kiện, tối ưu cho ứng dụng web thời gian thực. Nó là một nền tảng JavaScript thuần túy điều này có nghĩa là có thể sử dụng tất cả các kiến thức JavaScript để phát triển ứng dụng web. Node.js có thể xử lý nhiều yêu cầu cùng một lúc một cách hiệu quả [5].

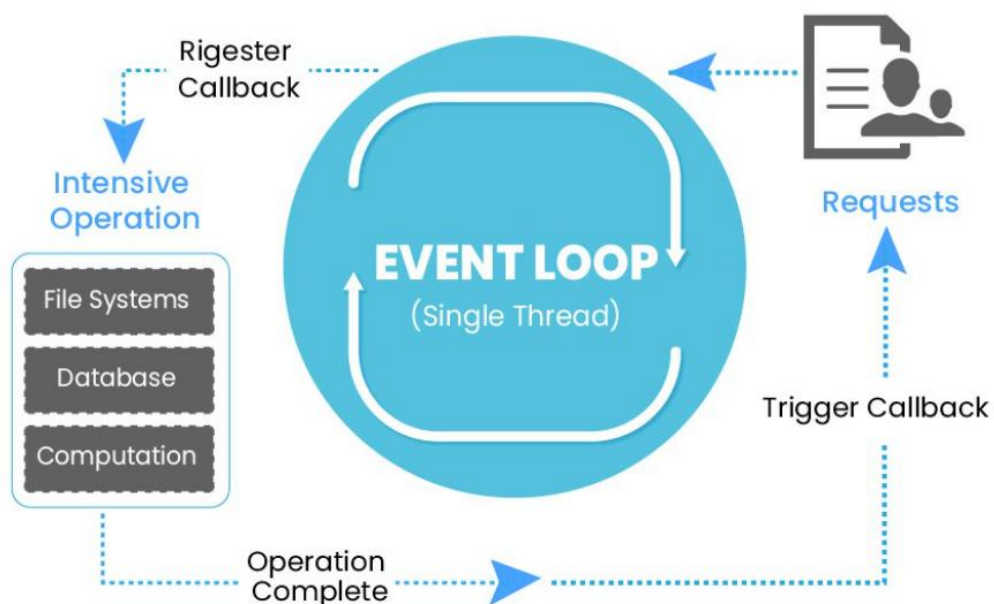


Hình 2.2: Các tính năng cơ bản của nodejs

(Nguồn: <https://www.linkedin.com/posts/sruthi2004-r-snsinstitutions-snsdesignthinkers-designthinking-activity-7194634022474452992-rnS5>)

2.3.1.2 Kiến trúc hoạt động của Node.js

Node.js hoạt động dựa trên một số nguyên tắc cơ bản giúp nó hiệu quả trong việc xử lý các ứng dụng có nhiều hoạt động nhập/xuất (I/O) mà không bị chặn, đồng thời giảm đáng kể sự phức tạp trong quản lý các luồng thực thi. Dưới đây là một số thành phần chính giải thích cách thức hoạt động của Node.js [5]:



Hình 2.3: Kiến trúc hoạt động của Node.js

(Nguồn: <https://vishwalakpriya99.medium.com/nodejs-1623cbdd3bfe>)

Kiến trúc Non-blocking I/O và Event-Driven: Node.js sử dụng một mô hình non-blocking I/O (input/output) và event-driven, nghĩa là các hoạt động như đọc file, truy vấn cơ sở dữ liệu, hoặc giao tiếp mạng được thực hiện mà không chặn tiến trình chính. Điều này cho phép xử lý nhiều yêu cầu cùng lúc mà không cần tạo nhiều luồng (thread), giúp giảm bớt chi phí liên quan đến quản lý luồng và tối ưu hóa hiệu suất. Khi một hoạt động I/O được khởi tạo, nó sẽ được gửi đến thực thi trong hệ thống hoặc cơ sở dữ liệu mà không làm chậm tiến trình chính. Sau khi hoạt động hoàn tất, một sự kiện sẽ được phát đi và xử lý bằng các hàm gọi lại (callback);

V8 JavaScript Engine: Node.js được xây dựng trên động cơ JavaScript V8 của Google Chrome, đây là một động cơ rất nhanh cho phép biên dịch mã JavaScript thành mã máy để thực thi trực tiếp trên phần cứng, làm tăng hiệu suất thực thi;

Single-Threaded: Mặc dù Node.js hoạt động trên một luồng duy nhất cho logic ứng dụng của người dùng, nó vẫn sử dụng nhiều luồng ở tầng thấp hơn thông qua thư viện libuv để xử lý các hoạt động I/O. Tuy nhiên, những chi tiết này được ẩn giấu khỏi người dùng, giúp việc lập trình đơn giản hơn mà vẫn đảm bảo hiệu suất;

Event Loop: Trái tim của Node.js là “event loop”. Đây là vòng lặp sự kiện mà ở đó Node.js tiếp tục lắng nghe sự kiện và thực hiện các hàm gọi lại khi một sự kiện được kích hoạt. Vòng lặp sự kiện cho phép Node.js xử lý hàng nghìn kết nối đồng thời mà không cần phải tạo ra chi phí quản lý luồng;

Trigger Callback: Khi thao tác I/O hoàn tất, hệ điều hành thông báo cho Node.js, và Node.js sau đó thực thi hàm callback tương ứng để xử lý kết quả hoặc tiếp tục xử lý logic;

NPM (Node Package Manager): NPM là hệ thống quản lý gói cho Node.js, cho phép các nhà phát triển dễ dàng chia sẻ và sử dụng mã nguồn từ nhau. NPM là một trong những kho lưu trữ mã nguồn mở lớn nhất thế giới và chứa hàng ngàn module có thể được tích hợp vào ứng dụng của bạn.

Require: Require thực hiện nhiệm vụ:

- Tải module đi kèm với Node.js như hệ thống file và HTTP từ Node.js API;

- Tải thư viện thứ 3 như Express và Mongoose mà bạn cài đặt từ npm;
- Giúp bạn require file của bạn và mô-đun hoá project;
- Require là 1 chức năng, và nó nhận tham số path tĩnh chính và trả về module.export.

2.3.2 Web Framework Express.js.

2.3.2.1 Giới thiệu về Express.js

Express.js là một framework web backend miễn phí và mã nguồn mở dành cho Node.js. Nó được thiết kế để xây dựng các ứng dụng web một trang, đa trang, và các API RESTful. Với Express.js, lập trình viên có thể tạo ra các ứng dụng web với kiến trúc mạnh mẽ, dễ bảo trì, và có khả năng mở rộng.

Express.js cung cấp một tập hợp các tính năng mạnh mẽ để phát triển ứng dụng web và API. Nó giúp quản lý định tuyến, xử lý request (yêu cầu) và response (phản hồi), cũng như tích hợp các middleware hiệu quả. Điều này làm cho việc phát triển web trở nên đơn giản hơn nhiều [6].

2.3.3 [1] Các tính năng của Epress.js

Sau đây là các tính năng nổi bật của ExpressJS [6]:

Thiết kế tối giản: ExpressJS là một thiết kế không có ý kiến. Framework ExpressJS rất linh hoạt với các nguyên tắc thiết kế đơn giản, tối giản. Chúng ta có thể nhanh chóng thiết lập máy chủ, xác định tuyến đường và bắt đầu xử lý các yêu cầu http với rất ít dòng mã;

Middleware/Routing: ExpressJS cung cấp các tùy chọn rõ ràng cho Routes với sự hỗ trợ cho tất cả các loại phương thức HTTP như POST, GET, PUT, v.v;

Tính linh hoạt: Không có điều kiện tiên quyết hoặc bất kỳ định dạng nào để sử dụng ExpressJS. Nếu chúng ta biết NodeJS, chúng ta có thể viết code theo sở thích của mình;

Template: ExpressJS hỗ trợ các công cụ tạo mẫu khác nhau như PUG, Jade hoặc EJS để tạo nội dung HTML một cách nhanh chóng;

Phục vụ tệp tĩnh: ExpressJS hỗ trợ các tệp tĩnh như images/css/javascript từ một thư mục một cách dễ dàng;

Dựa trên NodeJS : Các chức năng cốt lõi của lập trình không đồng bộ và các tính năng lập trình theo hướng sự kiện của NodeJS có thể được tích hợp với ExpressJS một cách dễ dàng.

2.4 Cơ sở dữ liệu và ORM

2.4.1 Hệ quản trị cơ sở dữ liệu MySQL

2.4.1.1 Giới thiệu về MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) mã nguồn mở, được sử dụng rộng rãi để lưu trữ, quản lý và truy xuất dữ liệu. MySQL hoàn toàn miễn phí nằm trong nhóm LAMP (Linux – Apache – MySQL – PHP).

Vì hệ quản trị cơ sở dữ liệu MySQL ổn định và dễ sử dụng, có tính khả chuyển, hoạt động trên nhiều hệ điều hành cung cấp một hệ thống lớn các hàm tiện ích rất mạnh và Mysql cũng có cùng một cách truy xuất và mã lệnh tương tự với ngôn ngữ SQL chính vì thế nên MySQL được sử dụng và hỗ trợ của những lập trình viên yêu thích mã nguồn mở [7].

Nhưng MySQL không bao quát toàn bộ những câu truy vấn cao cấp như SQL Server. Vì vậy MySQL chỉ đáp ứng việc truy xuất đơn giản trong quá trình vận hành của website, thích hợp cho các ứng dụng có truy cập CSDL trên internet và có thể giải quyết hầu hết các bài toán trong PHP, Perl.

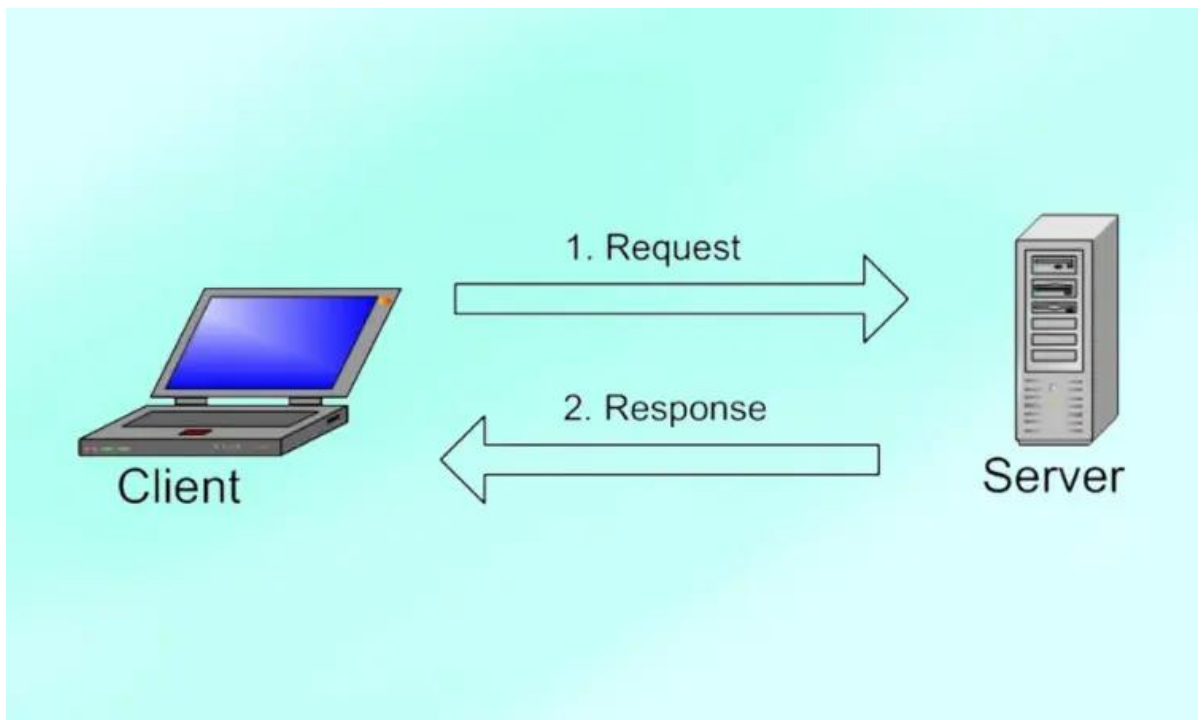
Các thuật ngữ phổ biến trong MySQL:

- Cơ sở dữ liệu (Database): Một tập hợp các dữ liệu có tổ chức được lưu trữ trong MySQL;
- Bảng (Table): Một phần quan trọng trong cơ sở dữ liệu, bảng chứa các hàng và cột để lưu trữ dữ liệu;
- Trường (Column): Là các đơn vị dữ liệu trong bảng, đại diện cho một thuộc tính cụ thể của dữ liệu;

- Hàng (Row): Một bản ghi cụ thể trong bảng, đại diện cho một bộ dữ liệu hoàn chỉnh;
- Khóa chính (Primary Key): Một trường duy nhất trong bảng, định danh duy nhất mỗi hàng trong bảng;
- Truy vấn (Query): Một câu lệnh SQL được sử dụng để truy xuất, thêm, sửa đổi hoặc xóa dữ liệu trong cơ sở dữ liệu;
- INDEX: Một cấu trúc dữ liệu được sử dụng để tăng tốc độ truy xuất dữ liệu trong cơ sở dữ liệu. Chúng giúp tìm kiếm dữ liệu nhanh hơn;
- Ràng buộc (Constraint): Một quy tắc được áp dụng cho dữ liệu trong bảng, đảm bảo tính nhất quán và hợp lệ của dữ liệu;
- Thủ tục lưu trữ (Stored Procedure): Một khối mã SQL được lưu trữ trong cơ sở dữ liệu và có thể được gọi để thực hiện một tác vụ cụ thể;
- Ghi nhật ký (Log): Ghi lại các hoạt động xảy ra trong cơ sở dữ liệu, giúp theo dõi và phục hồi dữ liệu khi cần thiết;
- Sao lưu và phục hồi (Backup and Restore): Quá trình tạo bản sao dự phòng của cơ sở dữ liệu để đảm bảo an toàn dữ liệu và khôi phục dữ liệu từ bản sao dự phòng khi cần;
- Truy cập từ xa (Remote Access): Cho phép kết nối và truy cập vào cơ sở dữ liệu MySQL từ xa thông qua mạng.

2.4.1.2 Cách hoạt động của MySQL

MySQL hoạt động theo mô hình client-server, nghĩa là có một máy chủ (server) lưu trữ phần mềm MySQL và các cơ sở dữ liệu, và có các máy khách (client) gửi yêu cầu đến máy chủ để thực hiện các thao tác với cơ sở dữ liệu [8].



Hình 2.3: Cách hoạt động của MySQL.

(Nguồn: <https://topdev.vn/blog/gioi-thieu-ve-mysql/>)

MySQL sử dụng ngôn ngữ SQL để thực hiện các tác vụ. Các câu lệnh SQL là các lệnh được gửi đến máy chủ MySQL để đọc, thao tác hoặc kiểm soát dữ liệu được lưu trữ trong các cơ sở dữ liệu của nó. Ví dụ, lệnh SELECT được sử dụng để lấy dữ liệu từ cơ sở dữ liệu. Để thêm dữ liệu mới, lệnh INSERT được sử dụng. Tương tự, các lệnh DELETE và UPDATE được sử dụng để xóa hoặc cập nhật dữ liệu hiện có.

Trong mô hình client-server, máy khách thiết lập kết nối với máy chủ thông qua mạng. Kết nối này cho phép người dùng truy cập vào các cơ sở dữ liệu trên server và thực hiện các thao tác trên chúng bằng cách sử dụng các câu lệnh SQL. Khi một request từ giao diện người dùng (Graphical user interface – GUI), server sẽ phản hồi thông tin và trả về kết quả trên máy client. Kỹ thuật ORM với Sequelize.

2.4.2 ORM

2.4.2.1 Giới thiệu về ORM

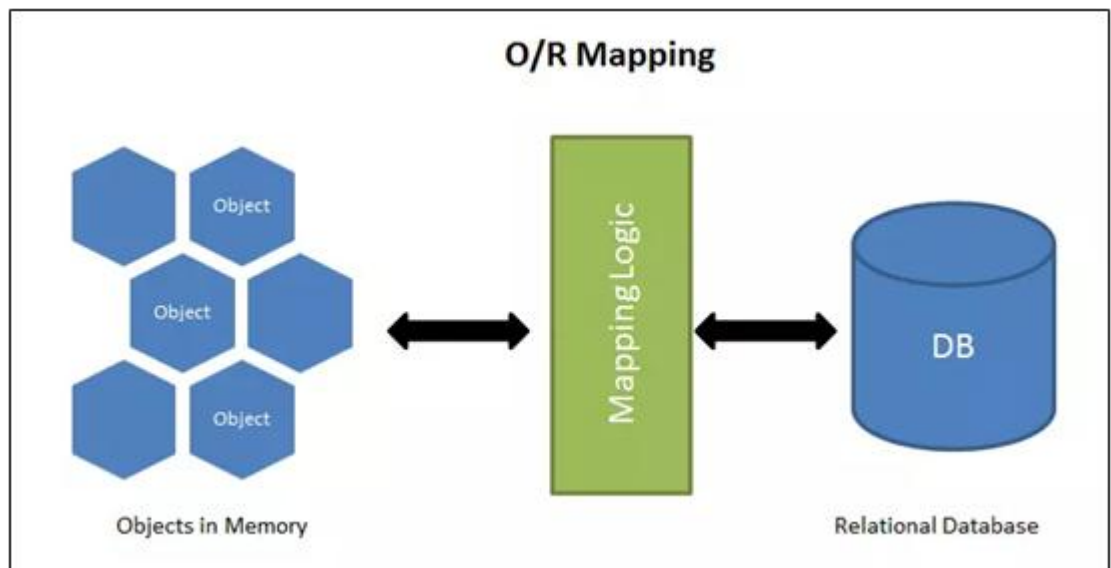
ORM là viết tắt của Object Relational Mapping. Nó là một kỹ thuật lập trình hoạt động như một trung gian giữa cơ sở dữ liệu quan hệ và một đối tượng của ngôn ngữ lập trình hướng đối tượng. Nó giảm thiểu nhu cầu viết các truy vấn SQL theo

cách thủ công và đơn giản hóa quá trình ánh xạ các đối tượng Java với các bảng cơ sở dữ liệu quan hệ, không giống như cách tiếp cận JDBC truyền thống [9].

Thực thể (lớp Java) -> Bảng (Cơ sở dữ liệu)

Trường -> cột

Mối quan hệ -> khóa ngoại



Hình 2.4: Mô hình Object–Relational Mapping (ORM)

(Nguồn: <https://viblo.asia/p/object-relational-mapping-djeZ1PQ3KWz>)

Đặc trưng cơ bản của ORM là gói gọn CSDL trong 1 object. 1 phần của object sẽ chứa data, và còn lại lo việc data xử lý như nào và biến nó thành CSDL quan hệ.

2.4.2.2 Ưu điểm của ORM

Sử dụng ORM đem đến những lợi ích cụ thể sau đây [9]:

Cải thiện năng suất: ORM sẽ tiến hành tự động hóa quá trình viết mã truy vấn nên giúp cho lập trình viên không cần phải ngồi viết câu lệnh SQL thủ công như trước. Nó không chỉ có tác dụng trong việc giảm thiểu thời gian phát triển mà còn giảm lỗi phát sinh có thể xảy ra khi viết thủ công. Hiện tại, ORM cũng đã được tích hợp trên nhiều framework phổ biến nên người dùng có thể dễ dàng xây dựng ứng dụng mà không cần xử lý cơ sở dữ liệu;

Độc lập với cơ sở dữ liệu: Đây là lợi ích được nhiều người chú ý khi sử dụng ORM. Bạn sẽ dễ dàng chuyển đổi những hệ quản trị cơ sở dữ liệu khác nhau mà

không phải thay đổi mã nguồn. Nhờ vậy việc phụ thuộc vào một loại cơ sở dữ liệu cũng được giảm thiểu và tăng khả năng linh hoạt trong quá trình bảo trì và quản lý hệ thống;

Khả năng bảo trì: ORM giúp lập trình viên có thể giảm thiểu tối đa lỗi có thể xảy ra trong quá trình viết câu lệnh SQL khi đánh máy;

Tính bảo mật cao: ORM cung cấp các phương pháp bảo mật tự động giúp ngăn chặn các lỗi phổ biến như SQL Injection. Bằng cách sử dụng các cơ chế chuẩn đề tương tác với cơ sở dữ liệu, ORM giúp đảm bảo rằng các truy vấn được thực hiện một cách an toàn và bảo mật. Điều này làm giảm thiểu rủi ro bảo mật và giúp bảo vệ dữ liệu của hệ thống;

Có thể tái sử dụng mã: Các phương thức và lớp đối tượng trong ORM thường tái sử dụng dễ dàng ở nhiều dự án khác nhau. Do đó, khi phát triển những ứng dụng mới người dùng cũng tiết kiệm công sức và thời gian hơn. Bên cạnh đó, bạn cũng có thể tạo ra những thư viện mã dùng chung để tăng tính đồng nhất và hiệu quả khi phát triển;

Tính nhất quán cao: Dữ liệu luôn đảm bảo tính nhất quán bằng việc kiểm soát thao tác và giao dịch trên dữ liệu. Nhất là với các hệ thống lớn việc đảm bảo tính nhất quán sẽ rất quan trọng giúp hệ thống đảm bảo tính ổn định.

2.4.2.3 Nhược điểm của ORM

Tính phức tạp cao: Sử dụng ORM có thể khiến code tăng thêm tính phức tạp và nhiều khi các lập trình viên sẽ chưa quen thuộc với điều đó;

Giảm hiệu suất trong một số trường hợp: Với một số trường hợp nhất định, các truy vấn SQL sẽ hoạt động kém hiệu quả hơn so với việc viết tay;

Hạn chế trong truy vấn SQL: Khi thực hiện các truy vấn phức tạp, ORM có thể gặp một số khó khăn. Những truy vấn ở dạng nâng cao, sử dụng những tính năng đặc biệt của SQL hay tối ưu hóa cụ thể của một loại cơ sở dữ liệu nó sẽ khó thực hiện hoặc không thể thực hiện với ORM.

2.5 Cơ chế xác thực bảo mật

2.5.1 Xác thực người dùng với JSON Web Token (JWT)

JSON Web Token (JWT) là một tiêu chuẩn mở (RFC 7519) định nghĩa một cách thức nhỏ gọn và khép kín để truyền tải thông tin an toàn giữa các bên dưới dạng đối tượng JSON. Thông tin này có thể được xác thực và tin cậy vì nó được ký bằng chữ ký số (Digital Signature) [10].

Cơ chế hoạt động: Trong mô hình xác thực phi trạng thái (Stateless Authentication), JWT hoạt động theo quy trình sau:

- Người dùng gửi thông tin đăng nhập (Username/Password) lên Server;
- Server xác thực thành công và trả về một chuỗi JWT (được ký bí mật);
- Client lưu trữ JWT (thường trong LocalStorage hoặc Cookie);
- Client gửi kèm JWT trong Header (Authorization: Bearer <token>). Server sẽ giải mã Token để xác định danh tính người dùng mà không cần truy vấn lại phiên làm việc (Session) trong cơ sở dữ liệu.

2.5.2 Mã hóa và bảo mật dữ liệu

Để đảm bảo tính toàn vẹn và bí mật của thông tin, các biện pháp mã hóa sau được áp dụng trong thiết kế hệ thống [11]:

Mã hóa mật khẩu (Password Hashing):

Nguyên tắc cơ bản trong bảo mật là mật khẩu người dùng không bao giờ được lưu trữ dưới dạng văn bản thuần (Plain-text). Thay vào đó, hệ thống sử dụng kỹ thuật băm mật khẩu một chiều kèm theo chuỗi ngẫu nhiên (Salt).

Thuật toán Bcrypt:

Hệ thống sử dụng thuật toán Bcrypt (được triển khai thông qua thư viện bcryptjs trong môi trường Node.js);

Hàm băm mật khẩu có độ an toàn cao, được thiết kế để thực hiện quá trình tính toán chậm có chủ đích;

Việc này làm chậm quá trình băm kết hợp với Salt giúp chống lại hiệu quả các cuộc tấn công vét cạn (Brute-force) và tấn công bằng bảng cầu vồng (Rainbow Table), đảm bảo mật khẩu gốc không thể bị giải mã ngay cả khi cơ sở dữ liệu bị lộ.

Bảo mật đường truyền:

Sử dụng giao thức HTTPS (Hypertext Transfer Protocol Secure) để mã hóa luồng dữ liệu truyền tải giữa Client (trình duyệt) và Server, ngăn chặn các hành vi nghe lén hoặc giả mạo dữ liệu trên đường truyền mạng.

CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU

3.1 Mô tả bài toán

Để giải quyết những bất cập trong công tác quản lý và xét duyệt học bổng thủ công hiện nay như tốn kém thời gian, dễ thất lạc hồ sơ và thiếu tính minh bạch, dự án này tập trung vào việc phát triển một nền tảng quản lý học bổng tập trung. Hệ thống đóng vai trò là cầu nối giữa ba đối tượng: Nhà trường, Sinh viên và Nhà tài trợ, giúp quy trình từ lúc công bố học bổng đến khi giải ngân diễn ra hoàn toàn trực tuyến và chính xác.

Các yêu cầu cụ thể của bài toán bao gồm:

– ***Kiến trúc đa trường (Multi-tenancy):***

Hệ thống phải có khả năng phục vụ nhiều trường đại học khác nhau trên cùng một nền tảng.

Dữ liệu của mỗi trường (sinh viên, học bổng, hồ sơ) phải được phân tách độc lập (Data Isolation), đảm bảo trường này không thể truy cập dữ liệu của trường kia.

– ***Quản lý chương trình học bổng:***

Cán bộ quản lý (Admin) có thể tạo mới, chỉnh sửa, đóng/mở các đợt học bổng theo từng học kỳ hoặc năm học.

Mỗi đợt học bổng cần lưu trữ đầy đủ thông tin: tên, số lượng suất, giá trị, hạn nộp hồ sơ và các tiêu chí xét duyệt đặc thù (GPA tối thiểu, điểm rèn luyện, hoàn cảnh khó khăn).

– ***Quy trình nộp và xét duyệt hồ sơ:***

Sinh viên: Có thể tìm kiếm học bổng phù hợp, nộp hồ sơ trực tuyến và tải lên các minh chứng (sổ hộ nghèo, bảng điểm). Hệ thống cần hỗ trợ theo dõi trạng thái hồ sơ (Chờ duyệt, Cần bổ sung, Đã duyệt, Từ chối) theo thời gian thực.

Cán bộ xét duyệt: Có công cụ để sàng lọc hồ sơ, xem xét minh chứng và thực hiện phê duyệt. Hệ thống cần có cơ chế "Lưu vết dữ liệu" (Data Snapshot) để đối chiếu thông tin sinh viên tại thời điểm nộp so với thời điểm hiện tại, tránh gian lận.

– ***Quản lý tài trợ và minh bạch tài chính:***

Hệ thống cho phép quản lý các nguồn quỹ từ Nhà tài trợ (Doanh nghiệp và Tổ chức).

Cung cấp chức năng báo cáo minh bạch: Nhà tài trợ có thể xem thống kê dòng tiền của mình đã được cấp cho những sinh viên nào (dưới dạng ẩn danh một phần thông tin nhạy cảm) để đảm bảo tính minh bạch.

– **Quản lý người dùng và phân quyền:**

+ **Hỗ trợ đăng nhập và xác thực bảo mật (JWT):** Phân quyền chặt chẽ theo vai trò: Quản trị hệ thống (Super Admin), Quản trị trường (Uni Admin), Sinh viên (Student) và Nhà tài trợ (Sponsor).

+ **Giao diện và trải nghiệm người dùng:** Xây dựng giao diện web thân thiện, tương thích trên cả máy tính và thiết bị di động, giúp sinh viên vùng sâu vùng xa với điều kiện mạng kém vẫn có thể tiếp cận dễ dàng.

3.2 Mô hình chức năng

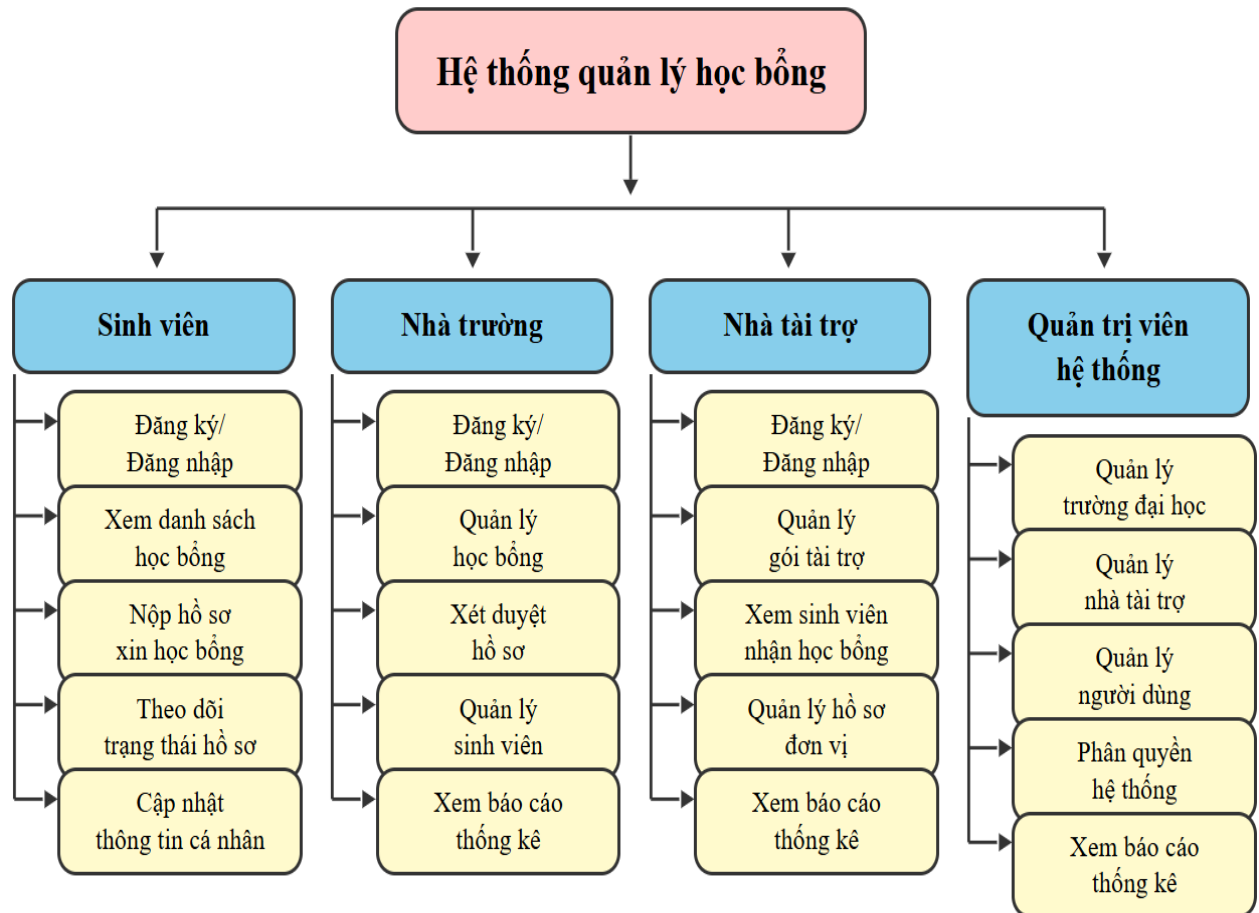
Sơ đồ chức năng hệ thống minh họa các vai trò và quyền hạn chính tương tác trong hệ thống quản lý học bổng, bao gồm 4 nhóm đối tượng: Sinh viên, Nhà trường, Nhà tài trợ và Quản trị viên hệ thống.

Sinh viên là đối tượng thụ hưởng chính, có thể thực hiện các hành động như đăng ký/đăng nhập, tìm kiếm và xem danh sách các gói học bổng phù hợp. Hệ thống cho phép sinh viên nộp hồ sơ trực tuyến, theo dõi trạng thái xét duyệt của hồ sơ theo thời gian thực và tự cập nhật thông tin cá nhân.

Nhà trường (Cán bộ quản lý) đóng vai trò vận hành quy trình cấp phát. Tạo lập và quản lý các đợt học bổng, thực hiện quy trình xét duyệt hồ sơ sinh viên gửi về, quản lý dữ liệu sinh viên trong trường và trích xuất các báo cáo thống kê phục vụ công tác quản lý.

Nhà tài trợ tham gia với vai trò đối tác hỗ trợ. Quản lý các gói tài trợ, cập nhật thông tin đơn vị và xem danh sách sinh viên đã nhận học bổng để đảm bảo tính minh bạch của dòng tiền tài trợ và chức năng xem báo cáo thống kê, giúp nhà tài trợ theo dõi hiệu quả giải ngân và dòng tiền tài trợ của đơn vị mình.

Quản trị viên hệ thống đảm nhiệm việc quản trị cấp cao nhất bao gồm việc quản lý danh sách các trường đại học và nhà tài trợ tham gia nền tảng. Chịu trách nhiệm quản lý toàn bộ người dùng và cấu hình phân quyền hệ thống và xem các báo cáo thống kê để giám sát quy mô hoạt động và sự ổn định của toàn bộ hệ thống



Hình 3.1 Mô hình chức năng

3.3 Phân tích yêu cầu hệ thống

3.3.1 Yêu cầu chức năng

Chức năng quản lý chương trình học bổng: Hệ thống cho phép tạo lập, chỉnh sửa, xóa và cập nhật trạng thái các đợt học bổng. Hỗ trợ tìm kiếm và lọc học bổng theo nhiều tiêu chí khác nhau.

Chức năng nộp và xét duyệt hồ sơ: Cung cấp quy trình nộp đơn trực tuyến khép kín. Người dùng có thể điền thông tin, tải lên các tài liệu minh chứng và theo dõi trạng thái hồ sơ theo thời gian thực.

Chức năng quản lý người dùng và phân quyền: Hỗ trợ đăng ký và đăng nhập tài khoản qua email an toàn. Hệ thống quản lý thông tin cá nhân người dùng,

đồng thời thực hiện phân quyền chặt chẽ theo vai trò và theo đơn vị trường), đảm bảo tính bảo mật dữ liệu giữa các đơn vị.

Chức năng báo cáo và thống kê: Tích hợp các công cụ phân tích số liệu, hiển thị biểu đồ thống kê về tình hình cấp phát học bổng.

3.3.2 Yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống cần đáp ứng các tiêu chuẩn kỹ thuật sau để đảm bảo hoạt động ổn định:

Hiệu năng:

Hệ thống phản hồi dưới 2 giây đối với các thao tác thông thường.

Tối ưu hóa tốc độ tải trang nhờ công nghệ Single Page Application (SPA) của ReactJS.

Bảo mật:

Sử dụng cơ chế xác thực JWT (JSON Web Token) để quản lý phiên làm việc an toàn.

Mật khẩu người dùng được mã hóa một chiều bằng thuật toán Bcrypt trước khi lưu vào cơ sở dữ liệu.

Đảm bảo phân tách dữ liệu giữa các trường đại học.

Khả năng tương thích:

Giao diện tương thích tốt trên nhiều loại thiết bị nhờ thiết kế Responsive.

Hoạt động ổn định trên các trình duyệt phổ biến hiện nay như Chrome, Firefox, Safari, Edge.

Khả năng bảo trì và mở rộng:

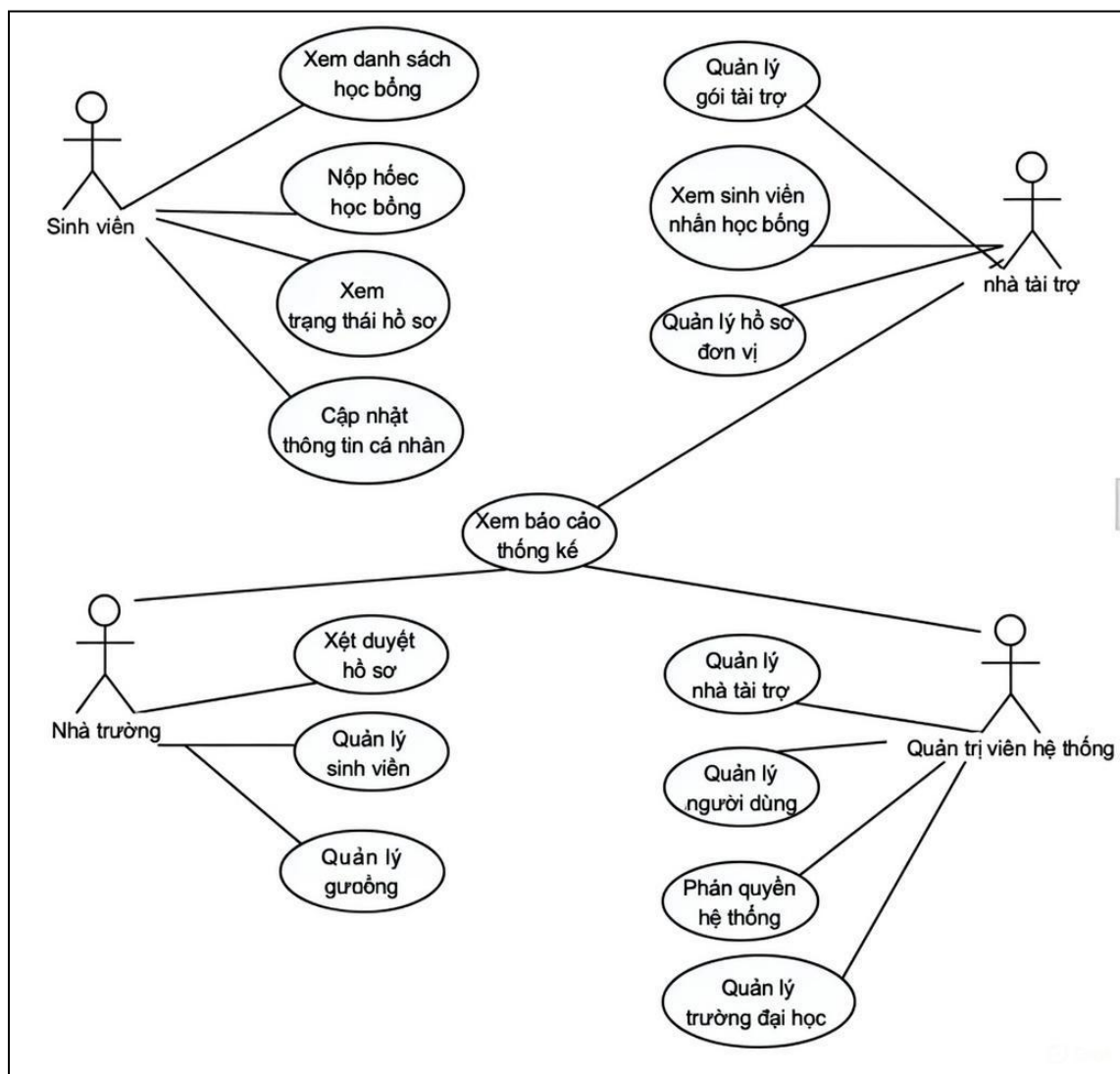
Mã nguồn được tổ chức theo mô hình MVC và Component-based và dễ dàng nâng cấp hoặc thêm tính năng mới.

Cơ sở dữ liệu được thiết kế chuẩn hóa, sẵn sàng mở rộng khi lượng dữ liệu tăng cao trong tương lai.

3.4 Thiết kế hệ thống

3.4.1 Sơ đồ Usecase

Dựa trên các yêu cầu đã phân tích sơ đồ Usecase xác định các tác nhân tham gia (Actor) bao gồm Sinh viên, Nhà trường, Nhà tài trợ và Quản trị viên hệ thống, đồng thời thể hiện mối quan hệ tương tác giữa các tác nhân này với các chức năng nghiệp vụ cụ thể nhằm đáp ứng yêu cầu quản lý.



Hình 3.2: Mô hình Usecase

Mô tả các Actor:

Bảng 3.1. Bảng mô tả các Actor.

STT	Tên Actor	Ý nghĩa
1	Sinh viên	Đối tượng thụ hưởng chính của hệ thống. Là sinh viên đang theo học tại các trường đại học tham gia hệ thống, có nhu cầu tìm kiếm và nộp hồ sơ xin học bổng.
2	Nhà trường	Đại diện cho phòng công tác sinh viên hoặc đơn vị quản lý học bổng tại trường đại học. Chịu trách nhiệm tạo lập các đợt học bổng, quản lý dữ liệu sinh viên và trực tiếp xét duyệt hồ sơ.
3	Nhà tài trợ	Đơn vị, tổ chức hoặc doanh nghiệp cung cấp nguồn tài chính cho quỹ học bổng. Tham gia hệ thống với vai trò giám sát, theo dõi sự minh bạch của dòng tiền và hiệu quả của các gói tài trợ.
4	Quản trị viên hệ thống	Người quản lý cấp cao nhất (chịu trách nhiệm vận hành nền tảng kỹ thuật, quản lý danh sách các trường đại học thành viên, cấp tài khoản quản trị và cấu hình hệ thống chung.

Mô tả các Usecase:

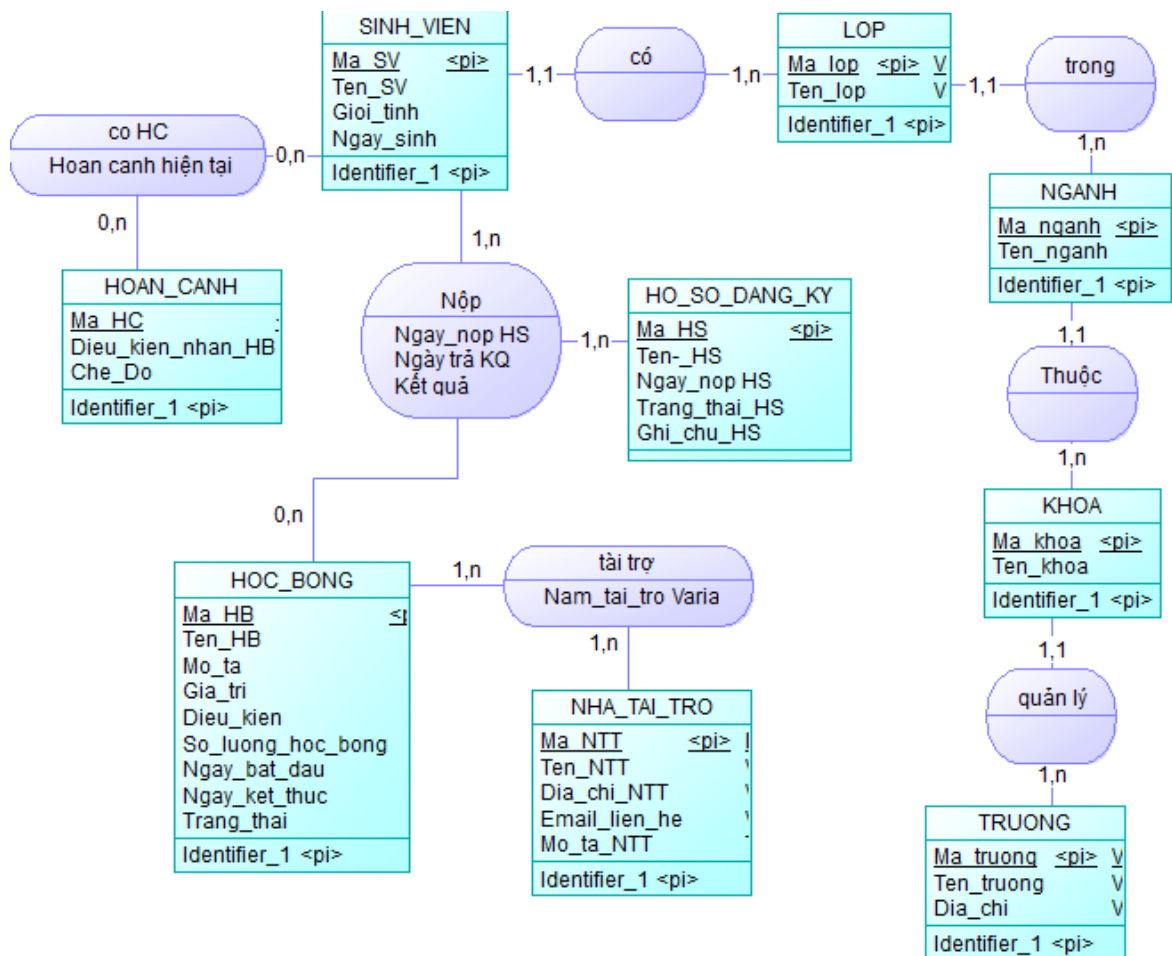
Bảng 3.2. Bảng mô tả các usecase.

STT	Tên usecase	Mô tả
1	Xem danh sách học bổng	Sinh viên xem danh sách các gói học bổng đang mở, tìm kiếm thông tin chi tiết về điều kiện và giá trị học bổng.
2	Nộp hồ sơ học bổng	Sinh viên thực hiện điền form đăng ký và tải lên các tài liệu minh chứng để ứng tuyển vào học bổng.
3	Xem trạng thái hồ sơ	Sinh viên theo dõi quá trình xử lý hồ sơ của mình (đang chờ, cần bổ sung, đã duyệt hay bị từ chối).

STT	Tên usecase	Mô tả
4	Cập nhật thông tin cá nhân	Sinh viên chỉnh sửa thông tin profile, thông tin liên lạc và tài khoản ngân hàng của bản thân.
5	Xem báo cáo thống kê	Chức năng chung cho Nhà trường, Nhà tài trợ và Quản trị viên để xem các biểu đồ số liệu về tình hình cấp phát và giải ngân.
6	Xét duyệt hồ sơ	Nhà trường xem chi tiết hồ sơ sinh viên, kiểm tra minh chứng và thực hiện thao tác duyệt hoặc từ chối.
7	Quản lý sinh viên	Nhà trường quản lý danh sách sinh viên thuộc trường mình, cập nhật thông tin hoặc import dữ liệu sinh viên.
8	Quản lý đợt học bổng	Nhà trường tạo mới, chỉnh sửa thông tin, cấu hình tiêu chí và đóng/mở các đợt xét tuyển học bổng.
9	Quản lý gói tài trợ	Nhà tài trợ tạo và quản lý các gói quỹ tài trợ, cam kết số tiền tài trợ cho các học bổng.
10	Xem sinh viên nhận học bổng	Nhà tài trợ xem danh sách các sinh viên đã được nhận tiền từ nguồn tài trợ của mình để đảm bảo minh bạch.
11	Quản lý hồ sơ đơn vị	Nhà tài trợ cập nhật thông tin giới thiệu, logo, website và thông tin liên hệ của tổ chức mình.
12	Quản lý nhà tài trợ	Quản trị viên hệ thống quản lý danh sách các đối tác tài trợ tham gia vào hệ thống.
13	Quản lý người dùng	Quản trị viên hệ thống cấp phát, khóa hoặc mở khóa tài khoản cho các cán bộ quản lý trường và đại diện nhà tài trợ.
14	Phân quyền hệ thống	Quản trị viên cấu hình vai trò và quyền hạn truy cập cụ thể cho từng nhóm người dùng trong hệ thống.
15	Quản lý trường đại học	Quản trị viên hệ thống thêm mới hoặc cập nhật thông tin các trường đại học tham gia sử dụng nền tảng.

3.4.2 Thiết kế cơ sở dữ liệu

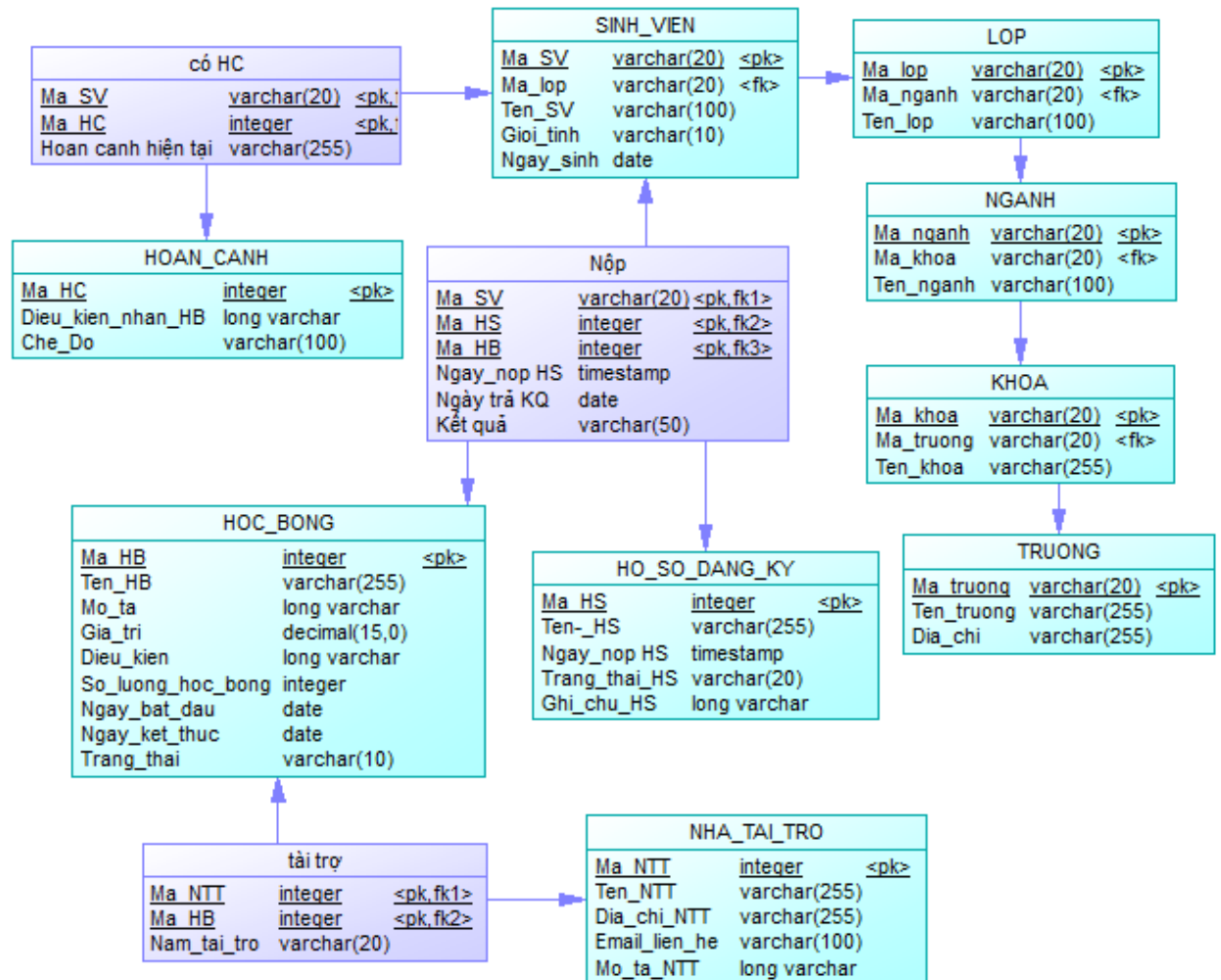
3.4.2.1 Mô hình thực thể kết hợp (ERD)



Hình 3.3: Mô hình dữ liệu quan hệ

Mô hình ERD giúp hiểu cấu trúc dữ liệu tổng quan của hệ thống, xoay quanh các thực thể cốt lõi như Sinh viên, Nhà trường, Nhà tài trợ và Học bổng. Các mối quan hệ được thiết kế nhằm đảm bảo tính toàn vẹn dữ liệu, từ việc quản lý thông tin hành chính đến quy trình nghiệp vụ nộp và xét duyệt hồ sơ đăng ký.

3.4.2.2 Mô hình dữ liệu vật lý



Hình 3.4: Mô hình dữ liệu vật lý

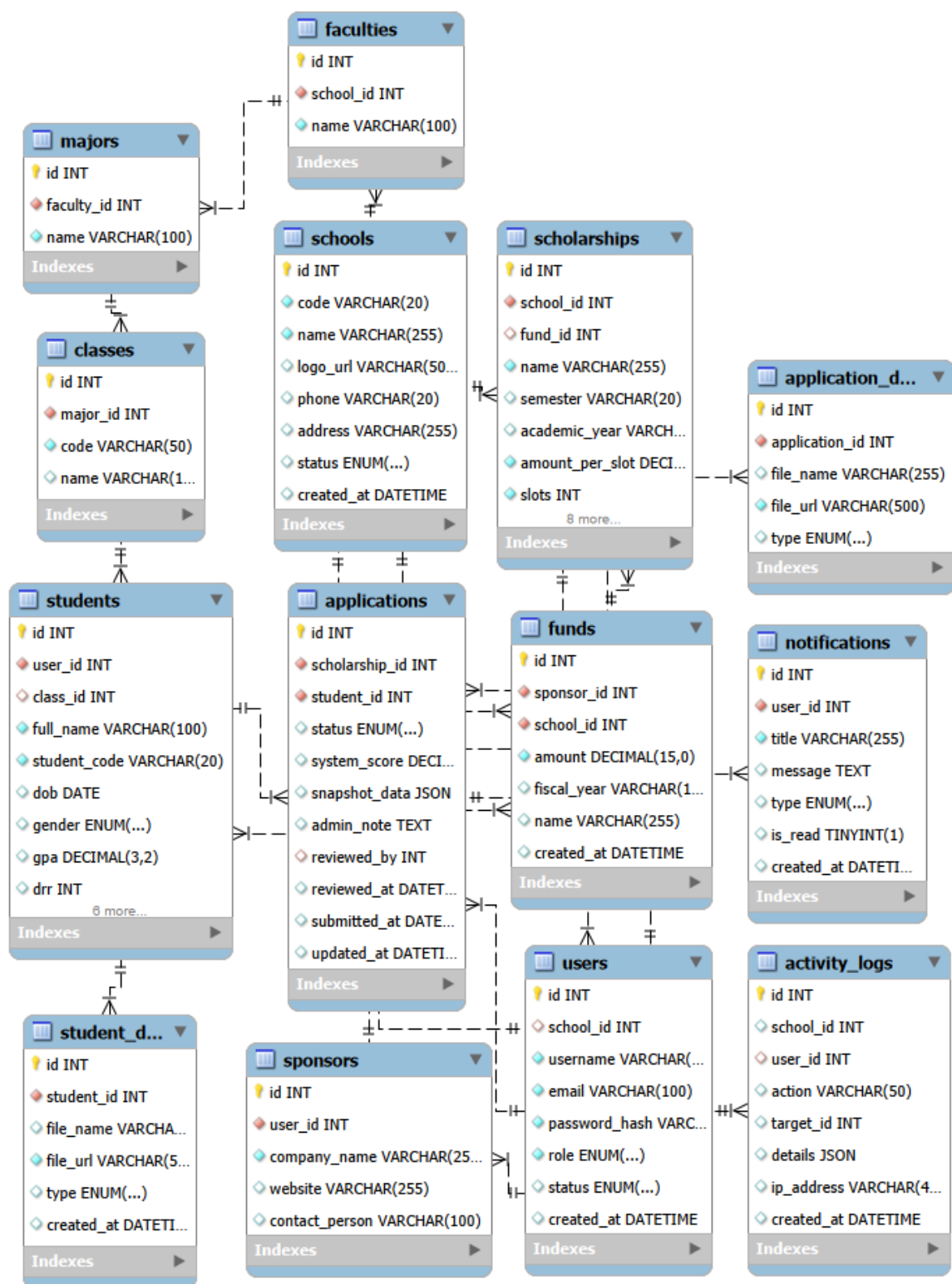
3.4.2.3 Mô hình dữ liệu quan hệ

Sau khi xác định các thực thể và mối quan hệ trong hệ thống quản lý học bổng từ đó xây dựng mô hình dữ liệu quan hệ để cài đặt vào hệ quản trị cơ sở dữ liệu MySQL.

1. SCHOOL (**Mã_truong**, Ten_truong, Mã_code, Logo_url, Dien_thoai, Dia_chi, Trạng_thai, Ngày_tao)
2. FACULTY (**Mã_khoa**, Ten_khoa, Mã_truong)
3. MAJOR (**Mã_nganh**, Ten_nganh, Mã_khoa)
4. CLASS (**Mã_lop**, Mã_code, Ten_lop, Mã_nganh)
5. USER (**Mã_người_dùng**, Ten_dang_nhap, Email, Mật_khau_hash, Vai_trò, Trạng_thai, Ngày_tao, Mã_truong)

6. STUDENT (**Ma sinh vien**, Ho_ten, Ma_so_SV, Ngay_sinh, Gioi_tinh, Diem_GPA, Diem_ren_luyen, Loai_ho_ngheo, So_tai_khoan, Ten_ngan_hang, Dien_thoai, Dia_chi, So_CCCD, *Ma_nguoi_dung, Ma_lop*)
7. SPONSOR (**Ma nha tai tro**, Ten_cong_ty, Ngươi_lien_he, Email_lien_he, Dien_thoai_lien_he, Dia_chi, Su_menh, Mo_ta, *Ma_nguoi_dung*)
8. FUND (**Ma quy**, So_tien, Nam_tai_chinh, Ten_quy, Ngay_tao, *Ma_nha_tai_tro, Ma_truong*)
9. SCHOLARSHIP (**Ma hoc bong**, Ten_hoc_bong, Hoc_ky, Nam_hoc, So_tien_moi_suat, So_suat, Mo_ta, Tieu_chi_JSON, Ngay_bat_dau, Ngay_ket_thuc, Trang_thai, Ngay_tao, Ngay_cap_nhat, *Ma_truong, Ma_quy, Ngươi_tao*)
10. APPLICATION (**Ma don**, Trang_thai, Diem_he_thong, Du_lieu_snapshot, Ghi_chu_admin, Ngay_duyet, Ngay_nop, Ngay_cap_nhat, *Ma_hoc_bong, Ma_sinh_vien, Ngươi_duyet*)
11. APPLICATION_DOCUMENT (**Ma tai lieu**, Ten_file, Duong_dan_file, Loai_tai_lieu, *Ma_don*)
12. STUDENT_DOCUMENT (**Ma tai lieu SV**, Ten_file, Duong_dan_file, Loai_tai_lieu, Ngay_tao, *Ma_sinh_vien*)
13. NOTIFICATION (**Ma thong bao**, Tieu_de, Noi_dung, Loai, Da_doc, Ngay_tao, *Ma_nguoi_dung*)
14. ACTIVITY_LOG (**Ma nhât ky**, Hanh_dong, Chi_tiet_JSON, Dia_chi_IP, Ngay_tao, *Ma_truong, Ma_nguoi_dung, Ma_doi_tuong*)

3.4.2.4 Lược đồ cơ sở dữ liệu.



Hình 3.5: Lược đồ cơ sở dữ liệu

Một số bảng chính trong dự án:

Bảng User (Người dùng)

Tên thực thể: User

Mô tả: Lưu trữ thông tin tài khoản đăng nhập của người dùng trong hệ thống.

Bảng 3.3. Mô tả bảng User.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	School_id	Integer	Khóa ngoại liên kết đến bảng School
3	Username	Varchar(50)	Tên đăng nhập
4	Email	Varchar(100)	Địa chỉ email
5	Password_hash	Varchar(255)	Mật khẩu đã mã hóa
6	Role	Enum	Vai trò
7	Status	Enum	Trạng thái tài khoản
8	Created_at	Datetime	Thời gian tạo tài khoản

Bảng School (Trường đại học)

Tên thực thể: School

Mô tả: Lưu trữ thông tin các trường đại học tham gia hệ thống.

Bảng 3.4. Mô tả bảng School.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	School_id	Integer	Mã trường
3	Username	Varchar(50)	Tên trường đầy đủ

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
4	Email	Varchar(100)	Đường dẫn hình ảnh logo trường
5	Password_hash	Varchar(255)	Số điện thoại liên hệ
6	Role	Enum	Địa chỉ trường
7	Status	Enum	Trạng thái hoạt động
8	Created_at	Datetime	Thời gian tạo

Bảng Student (Sinh viên)

Tên thực thể: Student

Mô tả: Lưu trữ thông tin hồ sơ chi tiết của sinh viên.

Bảng 3.5. Mô tả bảng Student.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	User_id	Integer	Khóa ngoại liên kết đến bảng User
3	Class_id	Integer	Khóa ngoại liên kết đến bảng Class
4	Full_name	Varchar(100)	Họ và tên sinh viên
5	Student_code	Varchar(20)	Mã số sinh viên
6	Dob	Date	Ngày sinh
7	Gender	Enum	Giới tính
8	Gpa	Decimal(3,2)	Điểm trung bình tích lũy
9	Drr	Integer	Điểm rèn luyện (thang điểm 100)
10	Poor_cert_type	Enum	Loại chứng nhận hoàn cảnh

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
11	Bank_number	Varchar(50)	Số tài khoản ngân hàng
12	Bank_name	Varchar(100)	Tên ngân hàng thụ hưởng
13	Phone	Varchar(15)	Số điện thoại cá nhân
14	Address	Varchar(255)	Địa chỉ thường trú
15	Id_number	Varchar(12)	Số Căn cước công dân

Bảng Sponsor (Nhà tài trợ)

Tên thực thể: Sponsor

Mô tả: Lưu trữ thông tin của các đơn vị, doanh nghiệp tài trợ.

Bảng 3.6. Mô tả bảng Sponsor.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	User_id	Integer	Khóa ngoại liên kết đến bảng User
3	Company_name	Varchar(255)	Tên công ty hoặc tổ chức tài trợ
4	Website	Varchar(255)	Địa chỉ website của đơn vị
5	Contact_person	Varchar(100)	Tên người đại diện liên hệ

Bảng Scholarship (Học bổng)

Tên thực thể: Scholarship

Mô tả: Lưu trữ thông tin các đợt/gói học bổng được mở.

Bảng 3.7. Mô tả bảng Scholarship.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
-----	----------------	--------------	---------

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	School_id	Integer	Khóa ngoại liên kết đến bảng School
3	Fund_id	Integer	Khóa ngoại liên kết đến bảng Fund
4	Name	Varchar(255)	Tên đợt học bổng
5	Semester	Varchar(20)	Học kỳ áp dụng
6	Academic_year	Varchar(20)	Năm học áp dụng
7	Amount_per_slot	Decimal(15,0)	Giá trị tiền cho mỗi suất học bổng
8	Slots	Integer	Tổng số lượng suất học bổng
9	Description	Text	Mô tả chi tiết về học bổng
10	Criteria_json	Json	Cấu hình tiêu chí xét duyệt
11	Start_date	Datetime	Ngày bắt đầu nhận hồ sơ
12	End_date	Datetime	Ngày kết thúc nhận hồ sơ
13	Status	Enum	Trạng thái
14	Created_by	Integer	ID của cán bộ tạo học bổng
15	Created_at	Datetime	Thời gian tạo
16	Updated_at	Datetime	Thời gian cập nhật lần cuối

Bảng Application (Hồ sơ xin học bổng)

Tên thực thể: Application

Mô tả: Lưu trữ thông tin hồ sơ đăng ký của sinh viên cho từng đợt học bổng.

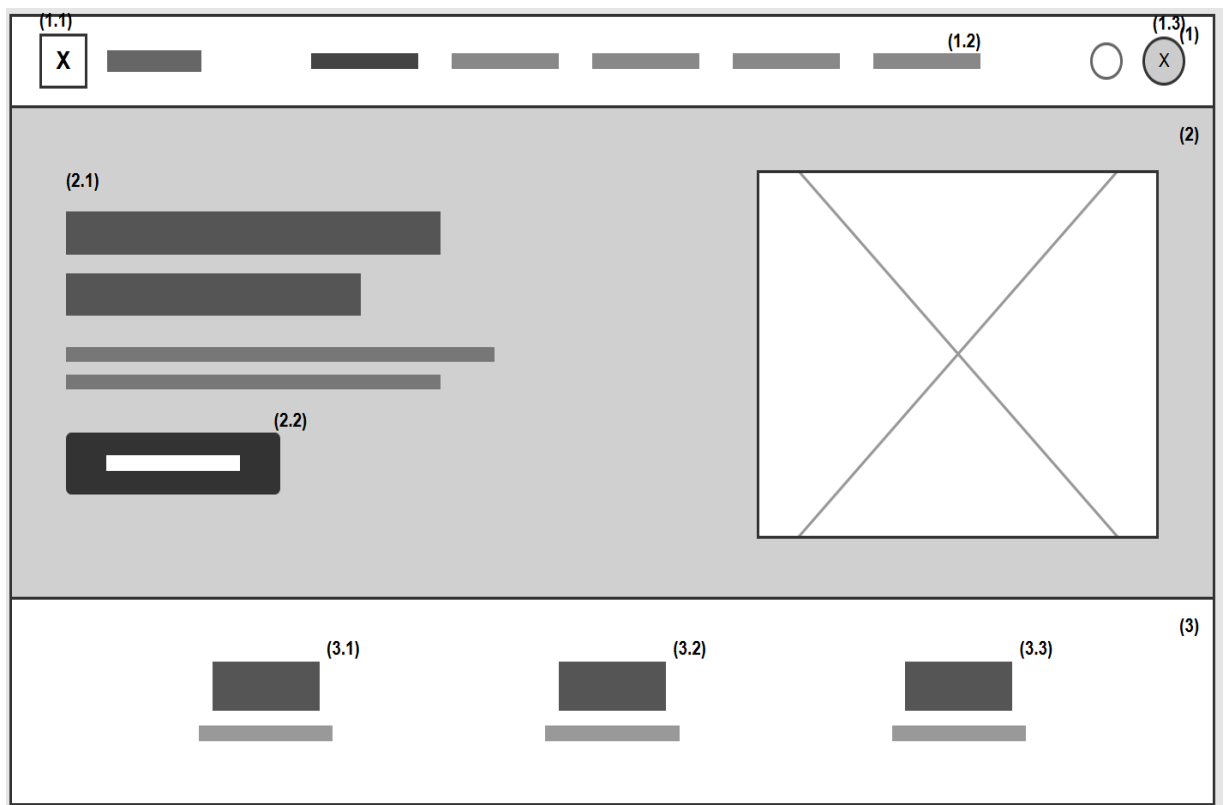
Bảng 3.8. Mô tả bảng Application.

STT	Tên thuộc tính	Kiểu dữ liệu	Ý nghĩa
1	Id	Integer	Khóa chính, tự động tăng
2	Scholarship_id	Integer	Khóa ngoại liên kết đến bảng Scholarship
3	Student_id	Integer	Khóa ngoại liên kết đến bảng Student
4	Status	Enum	Trạng thái hồ sơ
5	System_score	Decimal(5,2)	Điểm ưu tiên được hệ thống tính tự động
6	Snapshot_data	Json	Lưu trữ toàn bộ dữ liệu sinh viên tại thời điểm nộp
7	Admin_note	Text	Ghi chú của cán bộ xét duyệt
8	Reviewed_by	Integer	ID của cán bộ đã duyệt hồ sơ
9	Reviewed_at	Datetime	Thời gian thực hiện xét duyệt
10	Submitted_at	Datetime	Thời gian sinh viên nộp hồ sơ
11	Updated_at	Datetime	Thời gian cập nhật trạng thái

3.5 Thiết kế giao diện

3.5.1 Màn hình Low-fi prototypes trang chủ

Trang chủ (Homepage) được thiết kế theo phong cách Landing Page hiện đại, tập trung vào trải nghiệm người dùng với bố cục phân tầng rõ ràng từ trên xuống dưới. Mục tiêu là tạo ấn tượng về sự chuyên nghiệp, đồng thời điều hướng người dùng truy cập nhanh vào các chức năng cốt lõi.



Hình 3.6: Màn hình Low-fi prototypes trang chủ

(1) Thanh tiêu đề điều hướng (Header): Là khu vực cố định trên cùng, đóng vai trò định hướng không gian cho toàn bộ website, giúp người dùng duy trì khả năng truy cập các chức năng cốt lõi bất kể đang ở trang nào.

(1.1) Logo thương hiệu: Điểm neo thị giác giúp nhận diện hệ thống, đồng thời là phím tắt mặc định để quay về trang chủ.

(1.2) Hệ thống menu: Cung cấp bản đồ điều hướng phân cấp, giúp người dùng truy cập nhanh vào các phân hệ chức năng chính.

(1.3) Khu vực tiện ích cá nhân: Tập trung các tác vụ liên quan đến tài khoản và thông báo, phục vụ nhu cầu cá nhân hóa phiên làm việc.

(2) Khu vực Banner giới thiệu (Hero Section): Là vùng hiển thị trung tâm (Above-the-fold) có nhiệm vụ thu hút sự chú ý ngay lập tức và truyền tải giá trị cốt lõi của hệ thống.

(2.1) Thông điệp chính: Các dòng tiêu đề được phân cấp kích thước để truyền tải ngắn gọn mục đích và lợi ích của hệ thống.

(2.2) Nút kêu gọi hành động (Call-to-Action): Thành phần tương tác quan trọng nhất, được thiết kế nổi bật để điều hướng người dùng thực hiện hành vi mục tiêu (ví dụ: bắt đầu tìm kiếm).

(3) Khu vực Thông tin thống kê (Statistics Section): Vùng nội dung bổ trợ, sử dụng các số liệu thực tế để minh chứng cho quy mô và hiệu quả hoạt động.

(3.1), (3.2), (3.3) Các thẻ chỉ số (Social Proof): Hiển thị các con số thống kê tóm tắt (về số lượng đối tác, học bổng, sinh viên) nhằm xây dựng niềm tin và độ uy tín cho hệ thống đối với người truy cập.

3.5.2 Màn hình Low-fi prototypes danh sách học bổng

Giao diện danh sách học bổng được thiết kế với mục tiêu tối ưu hóa khả năng tìm kiếm và tra cứu thông tin. Bố cục sử dụng cấu trúc lưới (Grid Layout) quen thuộc, kết hợp với thanh công cụ lọc dữ liệu mạnh mẽ ở phía trên, giúp sinh viên nhanh chóng khoanh vùng và tiếp cận được các gói học bổng phù hợp nhất với nhu cầu giữa hàng trăm lựa chọn.



Hình 3.7: Màn hình Low-fi prototypes danh sách học bổng

(1) Thanh điều hướng chung (Header): Giữ nguyên cấu trúc đồng bộ với Trang chủ, đảm bảo người dùng luôn có thể truy cập các chức năng cốt lõi (Menu,

Thông báo, Tài khoản) mà không bị mất phương hướng khi di chuyển giữa các trang.

(2) Khu vực Tiêu đề & Thông báo kết quả: Hiển thị tên trang ("Danh sách học bổng") để định vị người dùng.

Quan trọng nhất là dòng thông báo số lượng kết quả tìm thấy cung cấp phản hồi tức thì về hiệu quả của việc tìm kiếm.

(3) Khu vực Bộ lọc và Tìm kiếm (Filter Bar): Đây là trung tâm điều khiển của trang, cho phép người dùng sàng lọc dữ liệu đầu vào.

(3.1) Ô tìm kiếm từ khóa: Nhập tên học bổng hoặc từ khóa liên quan.

(3.2) Bộ lọc Trường đại học: Dropdown cho phép chọn xem học bổng của một trường cụ thể (phục vụ tính năng Đa trường).

(3.3) Bộ lọc Trạng thái: Dropdown cho phép lọc theo tình trạng (Đang mở, Sắp hết hạn, Đã đóng).

(4) Khu vực Danh sách kết quả (Result Grid): Hiển thị danh sách các học bổng thỏa mãn điều kiện lọc dưới dạng các Thẻ thông tin (Cards) (4.1, 4.2, 4.3).

Mỗi thẻ được thiết kế để hiển thị tóm tắt các thông tin quan trọng nhất (Tên học bổng, Giá trị, Số suất, Hạn nộp), giúp người dùng so sánh nhanh và quyết định có bấm vào xem chi tiết hay không.

3.5.3 Màn hình Low-fi prototypes chi tiết học bổng

Giao diện chi tiết học bổng được thiết kế với mục tiêu minh bạch hóa thông tin, giúp sinh viên tự đánh giá mức độ phù hợp của bản thân.



Hình 3.8: Màn hình Low-fi prototypes chi tiết học bổng

(3) Khu vực Banner thông tin cốt lõi (Key Metrics Header): Đây là điểm nhìn đầu tiên, chứa Tên học bổng và các chỉ số quan trọng nhất được đóng khung riêng biệt để gây ấn tượng mạnh: Giá trị tài trợ, Tổng số suất và Số suất còn lại. Điều này giúp sinh viên quyết định nhanh xem học bổng có đủ hấp dẫn để đọc tiếp hay không.

(4) Khu vực Mô tả học bổng (Description): Cung cấp thông tin văn bản chi tiết về mục đích, đối tượng tài trợ và bối cảnh của học bổng, giúp sinh viên hiểu rõ ý nghĩa của chương trình.

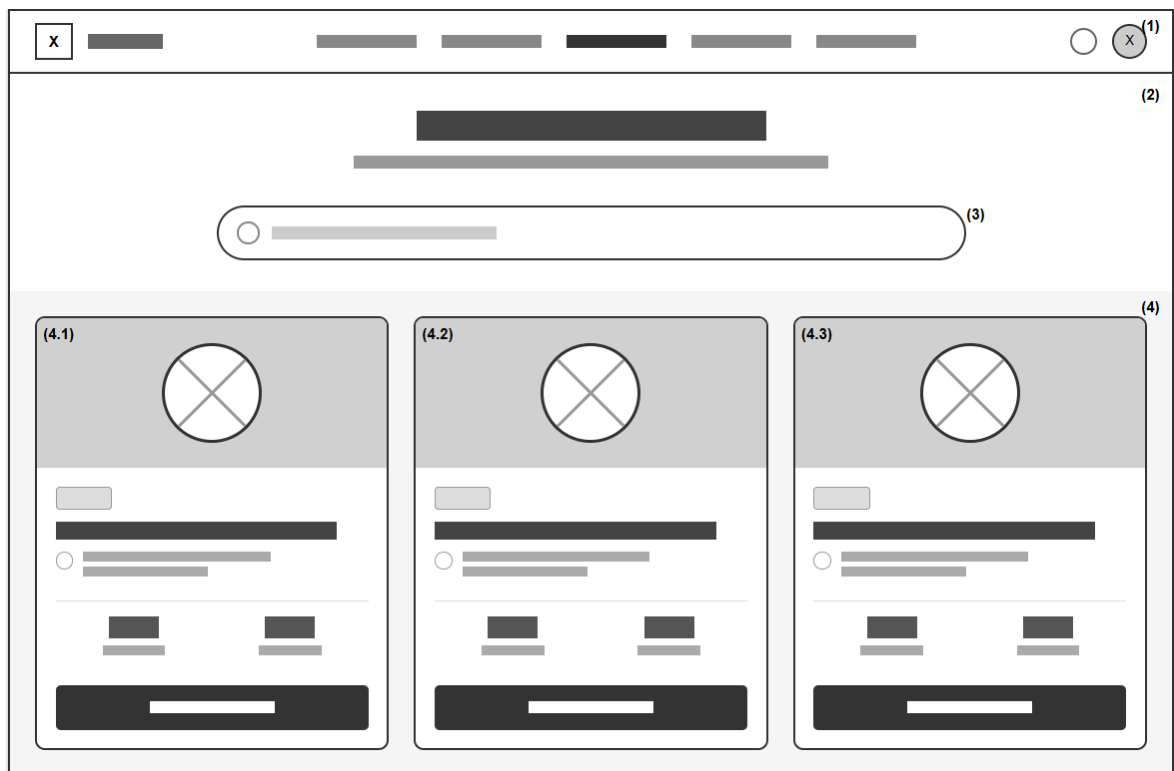
(5) Khu vực Tiêu chí xét duyệt (Criteria Checklist): Được thiết kế dưới dạng danh sách kiểm tra với các icon trực quan. Liệt kê rõ ràng các điều kiện bắt buộc như GPA tối thiểu, điểm rèn luyện, giấy chứng nhận hộ nghèo để sinh viên tự đối chiếu điều kiện của bản thân.

(6) và (7) Khu vực thông tin hỗ trợ: Nằm ở cột bên phải, cung cấp các thông tin về Thời gian (Ngày bắt đầu, Hạn chót nộp hồ sơ) và các ghi chú phụ. Vị trí này giúp thông tin luôn nằm trong tầm mắt nhưng không làm gián đoạn nội dung chính.

(8) Khu vực Hành động (Call-to-Action - CTA): Chứa nút "Nộp hồ sơ ngay". Đây là thành phần quan trọng nhất để chuyển đổi người xem thành người ứng tuyển. Được đặt nổi bật ở góc phải hoặc cố định khi cuộn trang, kèm theo lời nhắc nhở để thúc đẩy hành động.

3.5.4 Màn hình Low-fi prototypes danh sách các trường

Giao diện danh sách trường đại học được thiết kế để đóng vai trò như một cổng thông tin liên kết. Bố cục sử dụng thanh tìm kiếm nổi bật ở trung tâm và cách trình bày dạng lưới với các thẻ hình ảnh lớn, giúp sinh viên dễ dàng nhận diện thương hiệu của trường mình và truy cập nhanh vào các chương trình học bổng tương ứng.



Hình 3.9: Màn hình Low-fi prototypes danh sách các trường

(1) Thanh điều hướng chung (Header): Được giữ nguyên đồng bộ với toàn hệ thống, chứa các công cụ điều hướng thiết yếu và khu vực tài khoản cá nhân.

(2) Khu vực Tiêu đề & Giới thiệu: Hiển thị tên trang ("Danh sách Trường Đại học") và một dòng mô tả ngắn gọn về mục đích của trang (Khám phá các trường đối tác), giúp người dùng hiểu rõ ngữ cảnh sử dụng ngay khi truy cập.

(3) Thanh tìm kiếm trung tâm (Search Bar): Là thành phần tương tác chính, được thiết kế to, rõ ràng ở vị trí trung tâm. Cho phép người dùng nhập tên trường hoặc mã trường để lọc nhanh kết quả thay vì phải cuộn chuột tìm kiếm thủ công.

(4) Khu vực Danh sách Trường (University Grid): Hiển thị lưới các Thẻ thông tin trường (Cards) (4.1, 4.2, 4.3).

Cấu trúc một thẻ bao gồm:

Hình ảnh/Logo: Vị trí nổi bật nhất để nhận diện thương hiệu trường.

Thông tin định danh: Tên trường và Địa chỉ.

Số liệu thống kê: Hiển thị tóm tắt số lượng sinh viên và số học bổng đang mở (giúp sinh viên đánh giá độ "hot").

Nút hành động (CTA): Nút "Xem học bổng" ở dưới cùng để điều hướng người dùng sang trang chi tiết của trường đó.

3.5.5 Màn hình Low-fi prototypes hồ sơ cá nhân

Giao diện hồ sơ cá nhân được thiết kế để trở thành trung tâm quản lý dữ liệu định danh và học tập của sinh viên. Bố cục sử dụng các khối thông tin riêng biệt, phân tách rõ ràng giữa thông tin cá nhân và thông tin học tập được đồng bộ tự động giúp người dùng dễ dàng theo dõi và cập nhật.



Hình 3.10: Màn hình Low-fi prototypes hồ sơ cá nhân

(1) Thanh điều hướng chung (Header): Giữ nguyên đồng bộ với toàn hệ thống, giúp người dùng dễ dàng chuyển đổi giữa các chức năng.

(2) Khu vực Tóm tắt (Summary Card):

(2.1) Nút Đổi mật khẩu: Cho phép người dùng truy cập nhanh vào chức năng thay đổi thông tin đăng nhập để bảo vệ tài khoản.

(3) Khu vực Thông tin Định danh (Identity Section): Đây là phần nhận diện chính của sinh viên.

(3.2) Avatar & Tên: Hiển thị ảnh đại diện và tên đầy đủ, tạo cảm giác cá nhân hóa.

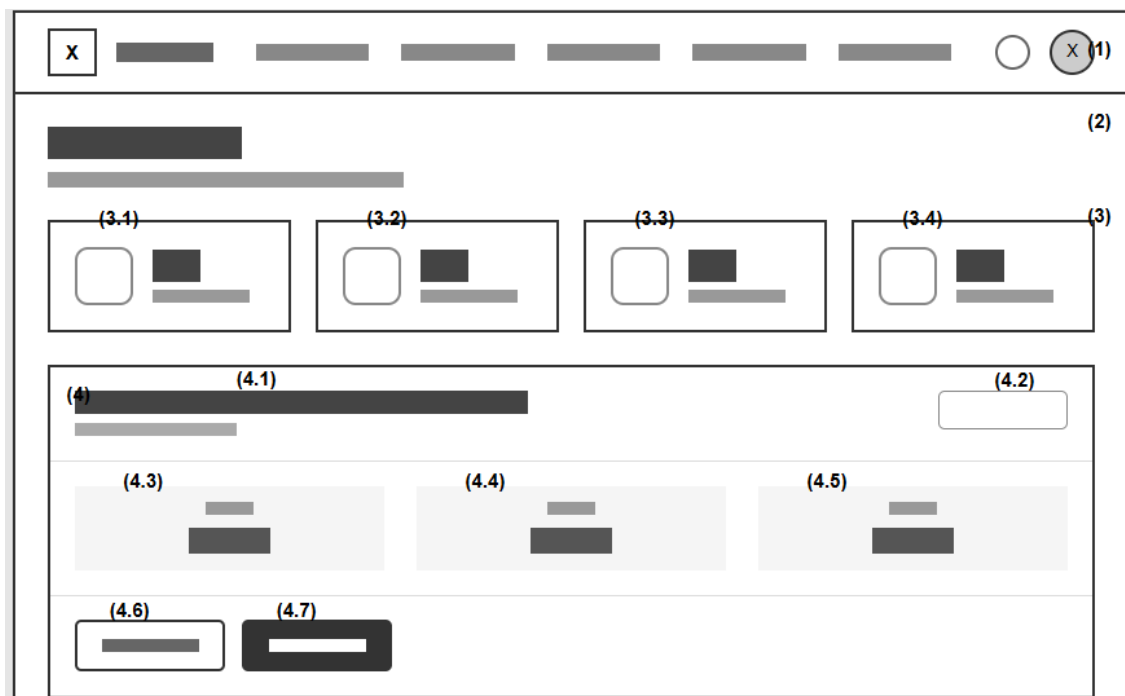
(3.3) Nút Chỉnh sửa: Cho phép sinh viên cập nhật các thông tin liên lạc (SĐT, Địa chỉ, Email phụ) để nhà trường liên hệ khi cần.

(4) Khu vực Thông tin Học tập (Academic Section): Hiển thị các chỉ số quan trọng dùng để xét duyệt học bổng.

(4.1), (4.2), (4.3), (4.4) Các trường dữ liệu: Hiển thị chi tiết về Điểm trung bình (GPA), Điểm rèn luyện, Lớp, Khoa. Các trường này thường ở chế độ "Chỉ xem" (Read-only) do được đồng bộ từ hệ thống đào tạo để đảm bảo tính chính xác và chống gian lận.

3.5.6 Màn hình Low-fi prototypes hồ sơ nộp học bổng của sinh viên

Giao diện theo dõi hồ sơ được thiết kế như một bảng điều khiển cá nhân tập trung vào việc hiển thị trạng thái xử lý của các đơn đăng ký theo thời gian thực. Bố cục phân tầng rõ ràng: tầng trên cùng là các chỉ số tổng hợp giúp người dùng nắm bắt nhanh tình hình chung, tầng dưới là danh sách chi tiết từng hồ sơ để người dùng thao tác cụ thể.



Hình 3.11: Màn hình Low-fi prototypes hồ sơ nộp học bổng của sinh viên

(1) Thanh điều hướng chung (Header): Giữ nguyên đồng bộ với toàn hệ thống.

(2) Khu vực tiêu đề: Hiển thị tên trang ("Hồ sơ của tôi") và thông điệp hướng dẫn ngắn gọn.

(3) Khu vực thống kê trạng thái (Status Summary): Giúp sinh viên không cần đếm thủ công.

(3.1) Tổng số hồ sơ: Tổng lượng đơn đã nộp.

(3.2) Chờ duyệt: Số lượng đơn đang trong hàng đợi xử lý.

(3.3) Đã duyệt: Số lượng đơn thành công (tín vui).

(3.4) Đã nhận tiền: Số lượng đơn đã hoàn tất giải ngân.

(4) Khu vực danh sách hồ sơ chi tiết (Application List): Hiển thị từng hồ sơ cụ thể dưới dạng thẻ lớn.

(4.1) Tên học bổng và ngày nộp: Thông tin định danh chính.

(4.2) Trạng thái hiện tại: Nhãn trạng thái (Tag) được mã hóa màu sắc (Vàng: Chờ, Xanh: Duyệt) để dễ nhận biết.

(4.3), (4.4), (4.5) Thông tin tóm tắt: Hiển thị lại các chỉ số quan trọng lúc nộp (GPA, ĐRL) và giá trị học bổng.

(4.6) Nút Xem chi tiết: Mở ra toàn bộ nội dung đơn đã nộp để xem lại.

(4.7) Nút Chỉnh sửa: Cho phép cập nhật lại thông tin hoặc bổ sung minh chứng (chỉ khả dụng khi hồ sơ ở trạng thái "Chờ duyệt" hoặc "Cần bổ sung").

CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU

4.1 Tổng quan kết quả

Sau quá trình nghiên cứu lý thuyết, phân tích yêu cầu và thiết kế hệ thống, đề tài "Xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn" đã được hiện thực hóa thành công. Hệ thống đã được xây dựng hoàn chỉnh theo kiến trúc Client-Server, đảm bảo sự tách biệt rõ ràng giữa Frontend và Backend.

Xây dựng thành công ứng dụng web hoạt động ổn định trên môi trường kiểm thử (Localhost/Docker). Hệ thống vận hành trơn tru với sự kết hợp của các công nghệ cốt lõi: ReactJS (Giao diện người dùng), Node.js và Express (Xử lý nghiệp vụ API), và MySQL (Lưu trữ dữ liệu quan hệ).

Hệ thống đã hoàn thiện các phân hệ chức năng quan trọng nhất theo đúng quy trình nghiệp vụ đã đề ra, bao gồm:

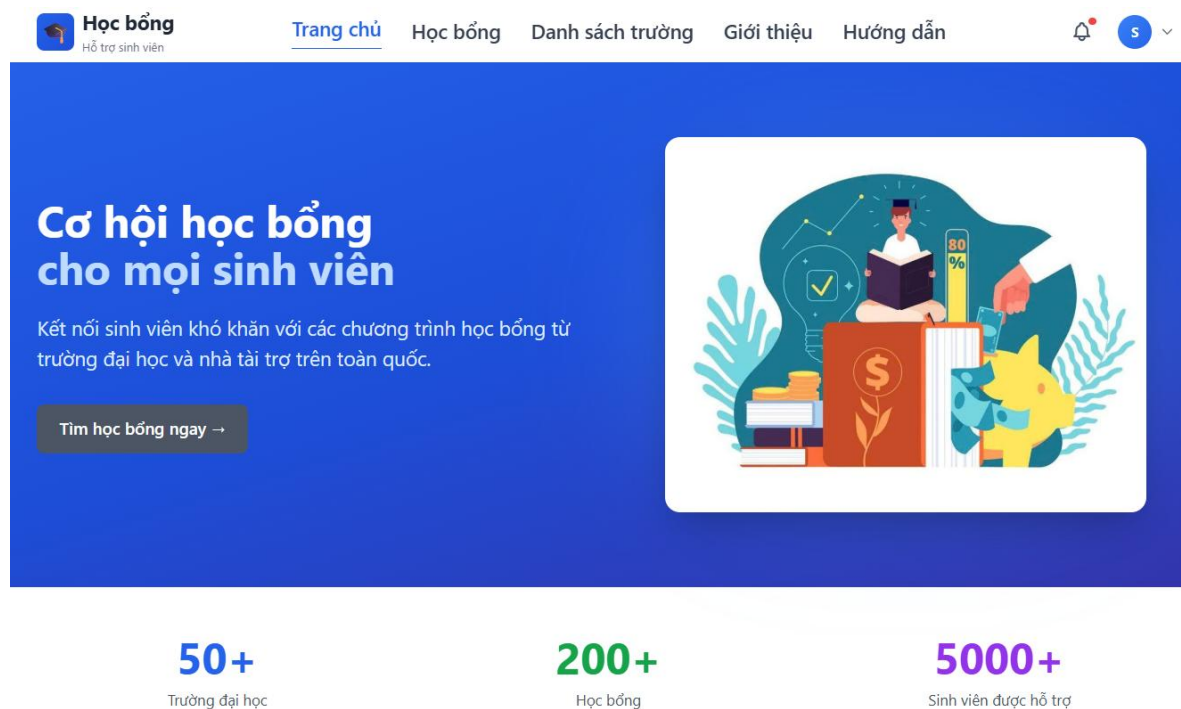
- Phân hệ Sinh viên: Cho phép tra cứu học bổng, nộp hồ sơ trực tuyến, tải minh chứng và theo dõi kết quả xét duyệt theo thời gian thực;
- Phân hệ Nhà trường (Uni Admin): Cung cấp công cụ toàn diện để quản lý các đợt học bổng, thực hiện quy trình xét duyệt hồ sơ, so sánh dữ liệu (Snapshot) và quản lý sinh viên;
- Phân hệ Quản trị hệ thống (Super Admin): Đóng vai trò trung tâm trong kiến trúc đa trường, cho phép quản lý danh sách các trường đại học tham gia hệ thống, cấp phát tài khoản quản trị cho từng trường và giám sát hoạt động chung của toàn bộ nền tảng;
- Phân hệ Nhà tài trợ: Đảm bảo tính minh bạch thông qua các báo cáo thống kê dòng tiền và danh sách thụ hưởng.

Áp dụng thành công các kỹ thuật nâng cao xác thực bảo mật bằng JWT (JSON Web Token), mã hóa mật khẩu và xuất báo cáo Excel chuẩn định dạng.

4.2 Giao diện người dùng

4.2.1 Giao diện trang chủ

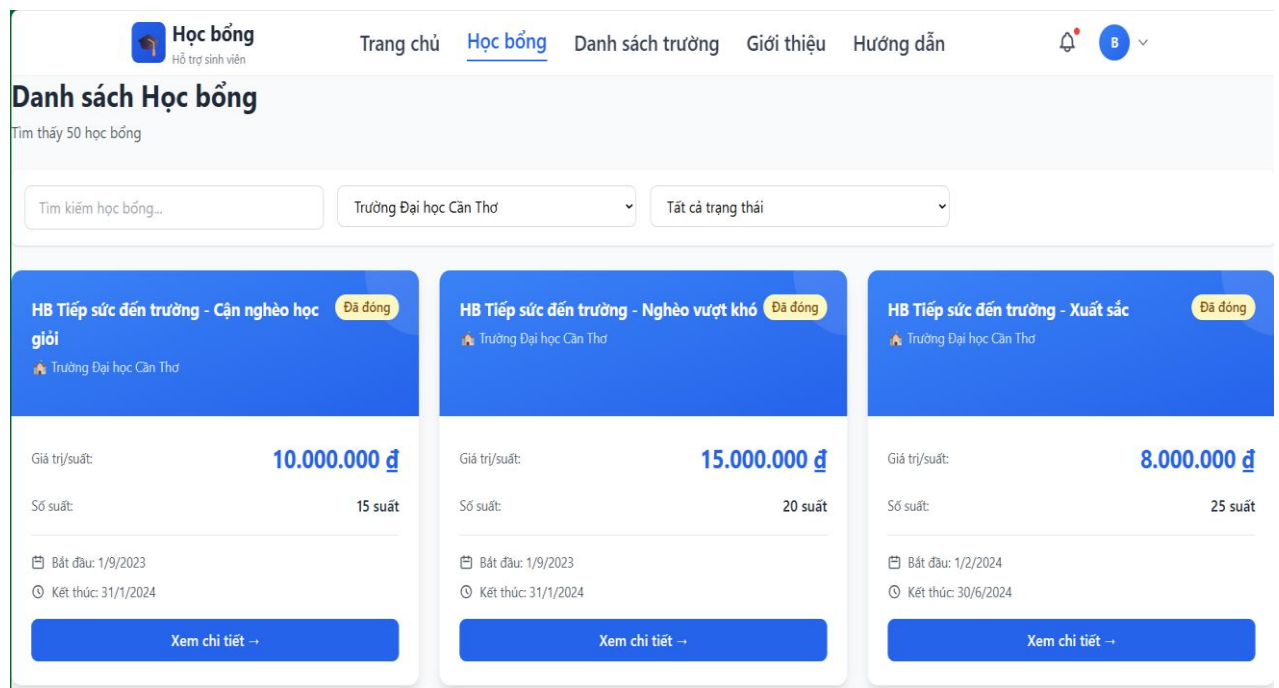
Trang tìm kiếm học bổng được thiết kế với thanh công cụ tìm kiếm và bộ lọc đa tiêu chí đặt ngay phần đầu trang, cho phép sinh viên dễ dàng sàng lọc kết quả theo tên trường, mức giá trị hỗ trợ hoặc trạng thái xét duyệt. Ngay bên dưới, danh sách các học bổng được trình bày trực quan dưới dạng thẻ (card), hiển thị tóm tắt các thông tin quan trọng như hạn nộp và số suất còn lại, giúp người dùng nhanh chóng nắm bắt và lựa chọn gói hỗ trợ phù hợp với hoàn cảnh của mình.



Hình 4.1 : Giao diện trang chủ

4.2.2 Giao diện học bổng

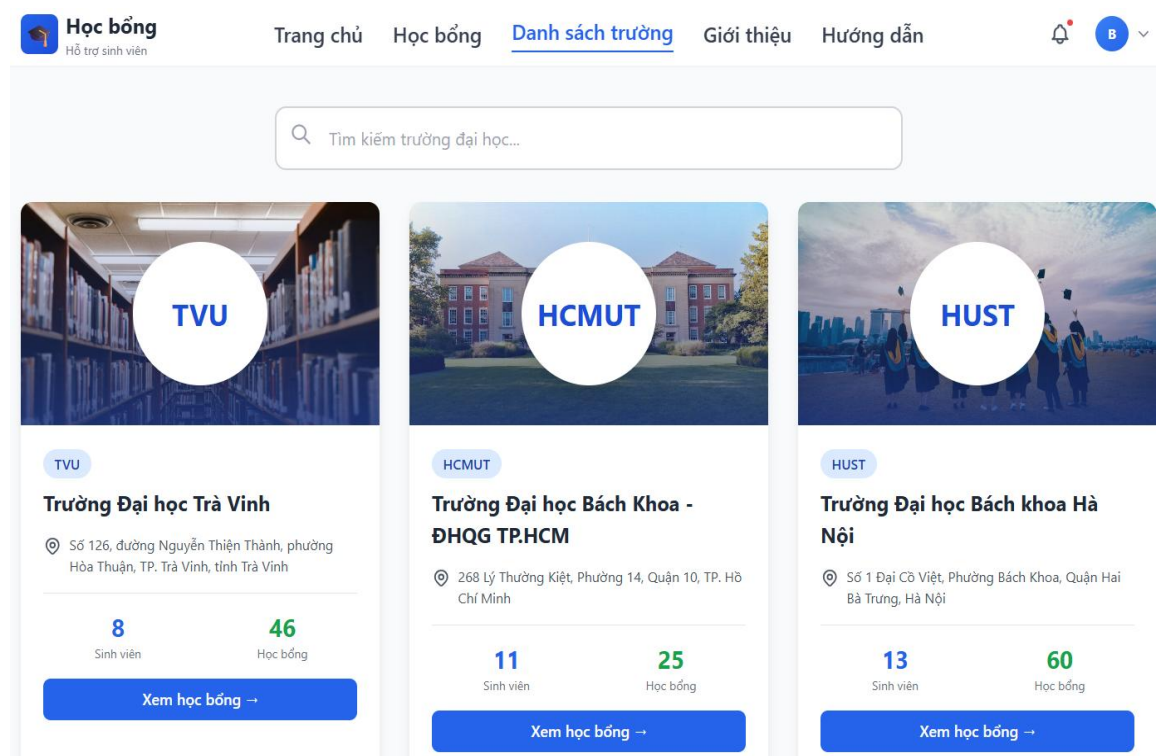
Giao diện tìm kiếm và lọc dữ liệu được thiết kế nhằm tối ưu hóa trải nghiệm tra cứu nhanh chóng khoanh vùng các cơ hội phù hợp giữa hàng trăm gói học bổng khác nhau. Cho phép lọc kết quả theo tên trường đại học, trạng thái cụ thể danh sách kết quả sẽ tự động cập nhật để hiển thị chính xác các gói tài trợ tương ứng dưới dạng thẻ trực quan, cung cấp đầy đủ các thông số quan trọng như giá trị, số suất và thời hạn nộp, giúp người dùng tiết kiệm thời gian và không bỏ lỡ các cơ hội tiềm năng.



Hình 4.2: Giao diện học bổng.

4.2.3 Giao diện danh sách các trường

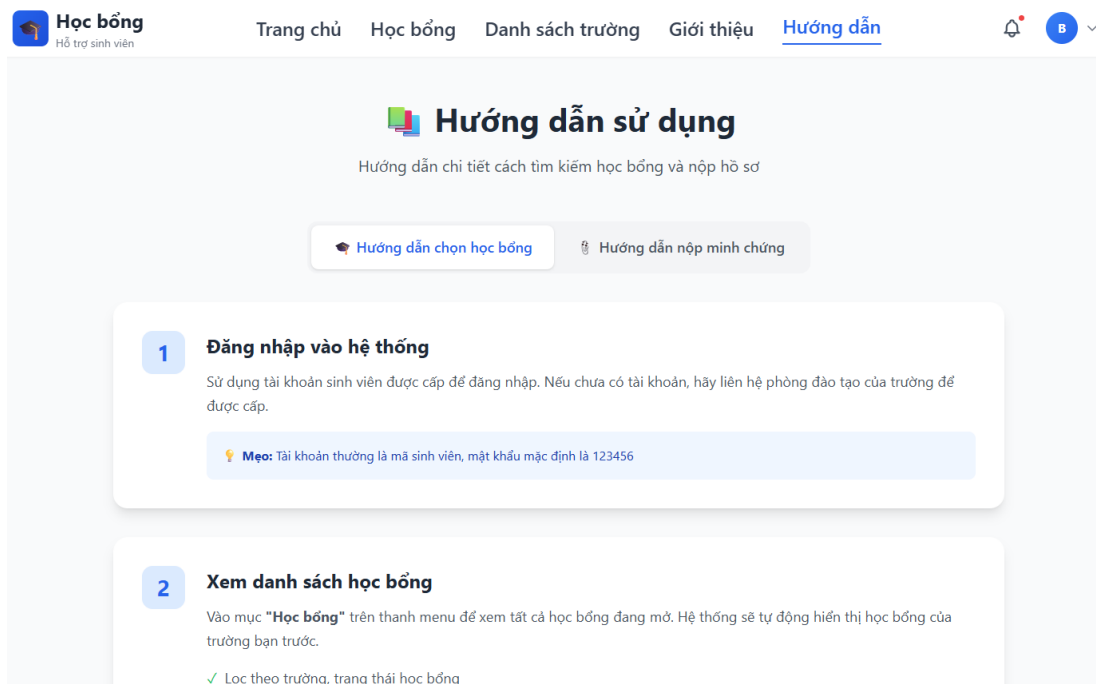
Giao diện danh sách trường đại học đóng vai trò là cổng thông tin liên kết, hiển thị mạng lưới các cơ sở giáo dục đối tác tham gia vào hệ thống. Trang web được thiết kế nhằm hỗ trợ sinh viên tra cứu và xác định nhanh đơn vị đào tạo của mình thông qua công cụ tìm kiếm trực tiếp và cách trình bày thông tin dạng thẻ trực quan. Đồng thời, chức năng điều hướng tích hợp trên từng thẻ cho phép người dùng đi sâu vào trang chi tiết của từng đơn vị để xem và ứng tuyển các gói học bổng đặc thù do riêng trường đó quản lý.



Hình 4.3: Giao diện danh sách các trường

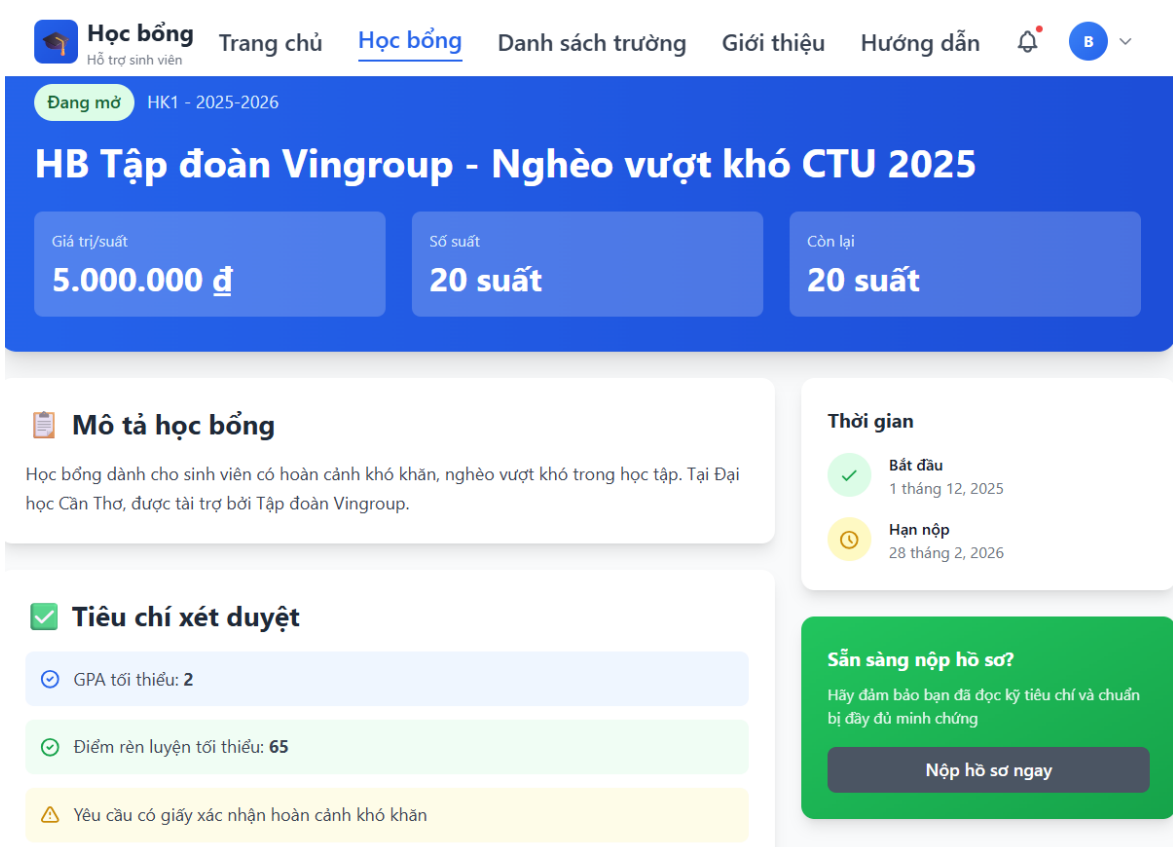
4.2.4 Giao diện hướng dẫn

Giao diện trang hướng dẫn cung cấp các quy trình thao tác chuẩn hóa nhằm giúp sinh viên, đặc biệt là người dùng mới, tiếp cận hệ thống một cách dễ dàng nhất chia nhỏ các tác vụ phức tạp thành các bước tuần tự, rõ ràng kèm theo hình minh họa. Và cho phép người dùng lựa chọn xem hướng dẫn chọn học bổng hoặc hướng dẫn nộp minh chứng tùy theo nhu cầu. Ngoài ra, các khung thông tin "Mẹo" và cảnh báo quan trọng được bố trí nổi bật để giải đáp trước các thắc mắc thường gặp, giúp giảm thiểu sai sót và nâng cao tỷ lệ nộp hồ sơ hợp lệ..



Hình 4.4: Giao diện hướng dẫn

4.2.5 Giao diện học bổng

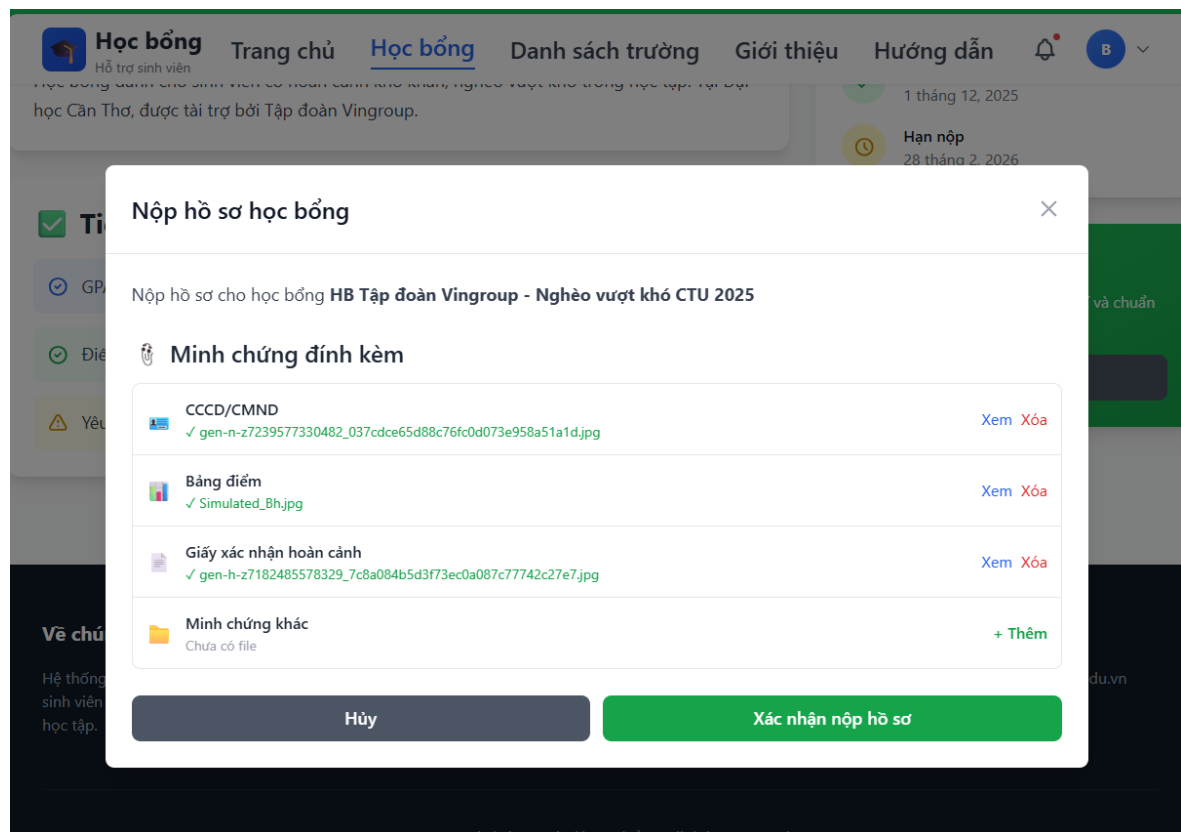


Hình 4.5: Giao diện học bổng

Giao diện chi tiết học bổng cung cấp cái nhìn toàn diện và minh bạch về từng gói hỗ trợ cụ thể. Hệ thống tập trung vào việc minh bạch hóa thông tin thông qua

việc trình bày rõ ràng các thông số cốt lõi như giá trị tài trợ, số lượng suất và các tiêu chí sàng lọc (GPA, điểm rèn luyện, hoàn cảnh), giúp sinh viên dễ dàng tự đánh giá năng lực bản thân. Bên cạnh đó, các mốc thời gian quan trọng về ngày mở và đóng đơn cũng được trực quan hóa nổi bật nhằm hỗ trợ người dùng nắm bắt lộ trình và chuẩn bị hồ sơ đúng hạn.

4.2.6 Giao diện nộp học bổng



Hình 4.6: Giao diện nộp học bổng

Giao diện nộp hồ sơ chức năng này là khả năng tự động hóa: hệ thống sẽ tự động trích xuất và đính kèm các tài liệu minh chứng (như CCCD, bảng điểm, giấy xác nhận hoàn cảnh) đã được lưu trong kho dữ liệu cá nhân của sinh viên vào đơn đăng ký. Sinh viên có quyền rà soát lại toàn bộ hồ sơ, xem trước nội dung file hoặc bổ sung thêm các minh chứng khác nếu cần thiết trước khi nhấn nút "Xác nhận", chính thức gửi dữ liệu vào quy trình xét duyệt của nhà trường.

4.2.7 Giao diện hồ sơ sinh viên

Học bổng
Hỗ trợ sinh viên

Trang chủ

Học bổng

Danh sách trường

Giới thiệu

Hướng dẫn

Hồ sơ của tôi

Chuẩn bị sẵn minh chứng để nộp học bổng nhanh hơn

Thông tin cơ bản

Họ và tên

Nguyễn Văn An

Mã sinh viên

B2101

Email

b2101@student.ctu.edu.vn

Số điện thoại

0945824907

Chỉnh sửa

Thông tin học tập

GPA

3.45

Điểm rèn luyện

85

* Thông tin học tập được đồng bộ từ hệ thống đào tạo

Minh chứng của tôi

Upload sẵn các minh chứng để sử dụng khi nộp hồ sơ học bổng

Chỉnh sửa

Hình 4.7: Giao diện hồ sơ sinh viên

Giao diện hồ sơ cá nhân được thiết kế một kho lưu trữ tập trung toàn bộ dữ liệu định danh và kết quả học tập của sinh viên, đóng vai trò là nguồn dữ liệu gốc để hệ thống tự động điền vào các mẫu đơn đăng ký. Cung cấp khả năng quản lý toàn diện cho phép sinh viên chủ động cập nhật thông tin liên lạc và theo dõi minh bạch các chỉ số học tập cốt lõi (GPA, điểm rèn luyện). Đặc biệt, tính năng kho lưu trữ số hóa cho phép người dùng tải lên và quản lý tập trung các minh chứng quan trọng (bảng điểm, giấy xác nhận, CCCD), giúp tối ưu hóa quy trình nộp hồ sơ bằng cách tái sử dụng dữ liệu cho nhiều đợt đăng ký khác nhau mà không cần thao tác lại từ đầu.

Huỳnh Phước Thọ

64

4.2.8 Giao diện hồ sơ đã nộp

Học bổng
Hỗ trợ sinh viên

Trang chủ Học bổng Danh sách trường Giới thiệu Hướng dẫn

Hồ sơ của tôi
Theo dõi trạng thái hồ sơ học bổng

Tổng hồ sơ **34**

Chờ duyệt **2**

Đã duyệt **30**

Đã nhận tiền **2**

Học bổng Phát triển Nguồn nhân lực ĐBSCL 2025
Nộp: 21/12/2025

GPA **3.45**

ĐRL **85**

Giá trị **12.000.000 đ**

Xem chi tiết Chỉnh sửa hồ sơ

Trợ cấp Xã hội theo Nghị định 81 năm học 2025-2026
Nộp: 14/12/2025

GPA **3.45**

ĐRL **85**

Giá trị **3.630.000 đ**

Xem chi tiết Chỉnh sửa hồ sơ

Hình 4.8: Giao diện hồ sơ đã nộp

Giao diện theo dõi hồ sơ đóng vai trò là trung tâm kiểm soát cá nhân, cung cấp cái nhìn tổng quan và minh bạch về toàn bộ lịch sử ứng tuyển của sinh viên. Với bố cục được chia thành khu vực thống kê trực quan và danh sách chi tiết, hệ thống cho phép người dùng nắm bắt nhanh tình trạng của các hồ sơ (từ Chờ duyệt, Đã duyệt đến Đã nhận tiền) thông qua các thẻ trạng thái được mã hóa màu sắc riêng biệt. Bên cạnh đó, giao diện tích hợp các chức năng tương tác sâu cho phép sinh viên xem lại chi tiết nội dung đã nộp hoặc thực hiện chỉnh sửa, bổ sung thông tin kịp thời đối với các hồ sơ đang trong giai đoạn xét duyệt, giúp họ chủ động kiểm soát hoàn toàn quy trình ứng tuyển của mình.

4.2.9 Giao diện thông tin cá nhân sinh viên

The screenshot shows a web interface for a student's profile. At the top, there is a navigation bar with the logo 'Học bổng' (Scholarship) and a sub-label 'Hỗ trợ sinh viên' (Support for students). The navigation menu includes 'Trang chủ' (Home), 'Học bổng' (Scholarship), 'Danh sách trường' (List of schools), 'Giới thiệu' (Introduction), 'Hướng dẫn' (Guidance), a notification bell, and a user profile icon labeled 'B'. The main content area has a blue header with a large white circle containing the letter 'N'. Below this, the student's name 'Nguyễn Văn An' is displayed, followed by the email 'b2101@student.ctu.edu.vn'. There are two tags: 'B2101' and 'Sinh viên' (Student). A 'Chỉnh sửa' (Edit) button is located to the right. The 'Thông tin liên hệ' (Contact information) section contains fields for 'Họ và tên' (Last name and first name) with the value 'Nguyễn Văn An', 'Email' with 'b2101@student.ctu.edu.vn', 'Số điện thoại' (Phone number) with '0945824907', and 'Địa chỉ' (Address) with 'khóm 4'. The 'Thông tin ngân hàng' (Bank information) section, with a subtitle '(Để nhận tiền học bổng)' (To receive scholarship money), contains fields for 'Tên ngân hàng' (Bank name) with 'VD: Vietcombank' and 'Số tài khoản' (Account number) with 'VD: 1234567890'.

Hình 4.9: Giao diện thông tin cá nhân sinh viên

Giao diện thông tin cá nhân đóng vai trò là nơi quản lý dữ liệu định danh trung tâm của sinh viên trong hệ thống các thông tin liên lạc thiết yếu (số điện thoại, địa chỉ) và đặc biệt là thông tin tài khoản ngân hàng để phục vụ cho quy trình giải ngân học bổng sau này. Để đảm bảo tính minh bạch và chính xác tuyệt đối trong khâu xét duyệt, các dữ liệu học tập quan trọng như GPA và điểm rèn luyện được thiết kế ở chế độ "chỉ xem" (read-only), được đồng bộ hóa trực tiếp từ hệ thống đào tạo của nhà trường và sinh viên không thể tự ý chỉnh sửa. Ngoài ra, các tính năng an toàn bảo mật như đổi mật khẩu cũng được tích hợp sẵn để người dùng bảo vệ tài khoản của mình.

4.3 Kết quả đạt được

Sau quá trình nghiên cứu và thực hiện đề tài "Xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn" đồ án đã hoàn thành các mục tiêu đề ra ban đầu.

Về mặt lý thuyết và công nghệ

– **Nắm vững quy trình nghiệp vụ:** Phân tích và mô hình hóa thành công quy trình xét duyệt học bổng phức tạp tại các trường đại học, từ khâu tiếp nhận hồ sơ, xét duyệt, đến giải ngân và báo cáo.

– **Làm chủ công nghệ hiện đại:** Ứng dụng thành công các công nghệ lập trình web tiên tiến bao gồm ReactJS (Frontend), Node.js và Express (Backend) và MySQL/Sequelize (Database).

– **Hiểu sâu về kiến trúc hệ thống:** Xây dựng thành công kiến trúc Multi-tenancy (Đa trường), cho phép hệ thống mở rộng để phục vụ nhiều đơn vị đào tạo trên cùng một nền tảng cơ sở dữ liệu.

– **Bảo mật hệ thống:** Áp dụng hiệu quả các chuẩn bảo mật như xác thực không trạng thái (JWT), mã hóa mật khẩu một chiều (Bcrypt) và phân quyền chặt chẽ (RBAC).

Về mặt sản phẩm thực nghiệm: Hệ thống đã được xây dựng hoàn thiện với các chức năng vận hành ổn định, đáp ứng nhu cầu của 4 nhóm đối tượng người dùng:

– **Đối với Sinh viên:**

+ Cung cấp một kênh thông tin chính thống, giúp sinh viên dễ dàng tra cứu và tiếp cận các nguồn hỗ trợ tài chính.

+ Đơn giản hóa thủ tục hành chính nhờ quy trình nộp hồ sơ và tải minh chứng hoàn toàn trực tuyến.

+ Giúp sinh viên chủ động theo dõi trạng thái hồ sơ theo thời gian thực, giảm thiểu sự lo lắng và chờ đợi.

– **Đối với Nhà trường (Cán bộ quản lý):**

+ Giảm tải 80% khối lượng công việc thủ công nhờ các công cụ lọc, tìm kiếm và sắp xếp hồ sơ thông minh.

+ Hỗ trợ công tác giải ngân nhanh chóng thông qua tính năng xuất báo cáo danh sách chuyển khoản ngân hàng chuẩn hóa.

– **Đối với Nhà tài trợ:** Đảm bảo tính minh bạch tài chính nhờ hệ thống báo cáo thống kê chi tiết. Nhà tài trợ có thể giám sát dòng tiền và xem danh sách sinh viên thụ hưởng thực tế.

– **Đối với Hệ thống:**

+ Giao diện người dùng (UI/UX) được thiết kế thân thiện, hiện đại và tương thích tốt trên nhiều thiết bị.

+ Hệ thống hoạt động ổn định, tốc độ phản hồi nhanh, đáp ứng tốt các yêu cầu phi chức năng về hiệu năng và trải nghiệm người dùng.

4.3.1 Đánh giá chung

Hệ thống đã giải quyết được các vấn đề cấp thiết trong công tác quản lý học bổng hiện nay như: sự thiếu minh bạch, quy trình thủ công rườm rà và khó khăn trong việc quản lý dữ liệu tập trung. Đây là tiền đề vững chắc để phát triển mở rộng hệ thống ứng dụng vào thực tế tại các cơ sở giáo dục

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận chung

Đề tài "Xây dựng hệ thống quản lý chương trình học bổng dành cho sinh viên khó khăn" đã hoàn thành xuất sắc các mục tiêu nghiên cứu và triển khai ban đầu, mang lại một giải pháp phần mềm toàn diện giúp số hóa quy trình xét duyệt và cấp phát học bổng tại các trường đại học. Sau quá trình phân tích nghiệp vụ, thiết kế hệ thống và kiểm thử thực nghiệm, đề tài đã đạt được những kết quả quan trọng trên nhiều phương diện:

Kiến trúc và cơ sở dữ liệu:

Đề tài đã xây dựng thành công mô hình cơ sở dữ liệu quan hệ MySQL được chuẩn hóa cao, đảm bảo tính toàn vẹn và nhất quán của dữ liệu thông qua việc ứng dụng kỹ thuật ORM với thư viện Sequelize. Điểm nổi bật nhất về mặt kỹ thuật là việc triển khai thành công kiến trúc Đa trường (Multi-tenancy), cho phép một hệ thống duy nhất có thể phục vụ đồng thời nhiều đơn vị đào tạo khác nhau. Cơ chế này không chỉ giúp tối ưu hóa tài nguyên phần cứng mà còn đảm bảo sự phân tách dữ liệu (Data Isolation) tuyệt đối, giúp thông tin của từng trường được bảo mật và quản lý độc lập. Cấu trúc mã nguồn của hệ thống được tổ chức khoa học theo mô hình MVC ở Backend và Component-based ở Frontend, tạo điều kiện thuận lợi cho việc bảo trì, sửa chữa và nâng cấp tính năng trong tương lai.

Khả năng xử lý nghiệp vụ và phân tích dữ liệu:

Hệ thống đã giải quyết triệt để các bài toán nghiệp vụ phức tạp đặc thù của quy trình xét duyệt học bổng. Cụ thể, cơ chế "Lưu vết dữ liệu" (Data Snapshot) đã được hiện thực hóa để lưu trữ trạng thái hồ sơ tại thời điểm nộp, giúp ngăn chặn gian lận và hỗ trợ công tác hậu kiểm chính xác. Bên cạnh đó, các thuật toán hỗ trợ ra quyết định như tự động xếp hạng ưu tiên (Auto-ranking) dựa trên GPA và hoàn cảnh khó khăn đã giúp giảm tải đáng kể áp lực cho cán bộ quản lý. Hệ thống cũng cung cấp khả năng truy vấn linh hoạt, cho phép trích xuất dữ liệu theo nhiều tiêu chí đa dạng và trực quan hóa các số liệu thông qua hệ thống báo cáo thống kê, giúp các nhà quản lý và nhà tài trợ có cái nhìn tổng quan về hiệu quả của dòng tiền hỗ trợ.

Tính ứng dụng và giá trị thực tiễn:

Sản phẩm của đề tài có tính ứng dụng cao, phù hợp với nhu cầu chuyển đổi số thực tế tại các cơ sở giáo dục. Hệ thống đáp ứng đầy đủ và hài hòa lợi ích của cả 4 nhóm đối tượng:

- Sinh viên được tiếp cận thông tin minh bạch và nộp hồ sơ dễ dàng; Nhà trường tối ưu hóa được quy trình quản lý hành chính;
- Nhà tài trợ giám sát được sự minh bạch của nguồn quỹ;
- Quản trị viên dễ dàng vận hành hệ thống.

Đặc biệt, các yêu cầu khắt khe về bảo mật thông tin cá nhân và dữ liệu tài chính ngân hàng đã được đảm bảo thông qua cơ chế xác thực JWT và mã hóa mật khẩu Bcrypt, tạo dựng niềm tin cho người sử dụng.

Hiệu quả công nghệ:

Việc lựa chọn và kết hợp các công nghệ lập trình hiện đại như ReactJS, Node.js và Express.js đã chứng minh được hiệu quả rõ rệt. Hệ thống vận hành mượt mà dưới dạng Single Page Application (SPA), mang lại trải nghiệm người dùng tốt với tốc độ phản hồi nhanh ngay cả khi xử lý lượng dữ liệu lớn. Khả năng mở rộng của công nghệ Node.js cũng đảm bảo hệ thống sẵn sàng tích hợp với các nền tảng khác của nhà trường hoặc mở rộng quy mô khi số lượng sinh viên tăng lên trong tương lai.

5.2 Hạn chế

Do thời gian thực hiện đề tài có hạn, cơ sở dữ liệu chưa được triển khai thực tế tại các trường học để đánh giá hiệu quả và tính thực tiễn trong môi trường thực tế cần được cải thiện và khắc phục trong tương lai.

5.3 Hướng phát triển

Mặc dù hệ thống đã hoàn thiện các chức năng cơ bản và vận hành ổn định, nhưng để đáp ứng nhu cầu ngày càng cao của công tác chuyển đổi số trong giáo dục đề tài định hướng phát triển trong tương lai tập trung vào các nội dung sau:

Tích hợp AI gợi ý học bổng: Sử dụng các thuật toán Học máy (Machine Learning) để phân tích hồ sơ sinh viên (GPA, hoàn cảnh, hoạt động ngoại khóa) và tự động đề xuất các gói học bổng có tỷ lệ trúng tuyển cao nhất cho từng cá nhân.

Chatbot hỗ trợ trực tuyến: Tích hợp Chatbot AI để tự động giải đáp các thắc mắc thường gặp của sinh viên về quy trình nộp hồ sơ, tiêu chí xét duyệt 24/7, giảm tải cho cán bộ tư vấn.

Kết nối hệ thống đào tạo: Tích hợp sâu hơn với hệ thống Quản lý đào tạo (LMS/SIS) của các trường đại học thông qua API để tự động đồng bộ điểm số và hạnh kiểm theo thời gian thực, loại bỏ hoàn toàn việc sinh viên phải nộp bảng điểm thủ công.

Mở rộng mạng lưới: Nâng cấp hạ tầng Server và cơ sở dữ liệu để hệ thống có thể phục vụ quy mô cấp Quốc gia, kết nối hàng trăm trường đại học và các quỹ thiện nguyện lớn trên toàn quốc.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] M. Massé, REST API Design Rulebook, 1, Ed., Sebastopol, CA: O'Reilly Media, Inc., 2012.
- [2] C. L. V. Tiến, "RESTful API là gì? Nguyên tắc, cách thức hoạt động, thiết kế RESTful API chuẩn," Vietnix, 21 July 2025. [Online]. Available: <https://vietnix.vn/restful-api-la-gi/>.
- [3] A. B. a. E. Porcello, Learning React: Functional Web Development with React and Redux, Sebastopol, CA: O'Reilly Media, Inc., 2017.
- [4] N. Đ. Hòa, "Tailwind CSS Là Gì? Cách cài đặt và sử dụng Tailwind CSS chi tiết," LANIT JSC, 9 9 2025. [Online]. Available: <https://lanit.com.vn/tailwind-css-la-gi.html>. [Accessed 13 12 2025].
- [5] B. Semah, "What Exactly is Node.js? Explained for Beginners," freeCodeCamp, 5 12 2022. [Online]. Available: <https://www.freecodecamp.org/news/what-is-node-js/?referrer=grok.com>. [Accessed 18 12 2025].
- [6] E. M. Hahn, Express in Action: Writing, Building, and Testing Node.js Applications, Shelter Island, NY: Manning Publications Co., 2016.
- [7] Ý. Nhi, "MySQL là gì? Tìm hiểu toàn tập về MySQL cho người mới bắt đầu," Miko Tech, 16 5 2023. [Online]. Available: <https://mikotech.vn/mysql-la-gi/?referrer=grok.com>. [Accessed 18 12 2025].
- [8] D. Durante, Supercharging Node.js Applications with Sequelize, Birmingham, U.K.: Packt Publishing, 2022.
- [9] M. Scofield, "Object Relational Mapping," 17 1 2020. [Online]. Available: <https://viblo.asia/p/object-relational-mapping-djeZ1PQ3KWz>. [Accessed 20 12 2025].
- [10] S. E. Peyrott, The JWT Handbook, Bellevue, WA: Auth0 Inc., 2017.
- [11] K. Dũüna, Secure Your Node.js Web Application, Dallas, TX: The Pragmatic Bookshelf, 2016.